Walking Routines, Everyday Activity, and Fear- and Anxiety-Related Behavior in Golden Retrievers: A Longitudinal Mixed-Effects Analysis

Alisa Tananaeva
March 2026

Abstract

Fear- and anxiety-related behavior problems are common in dogs, yet their longitudinal dynamics and relationship to everyday routines remain insufficiently understood. This study assessed year-to-year stability in fear- and anxiety-related outcomes and evaluated the predictive value of walking and activity routines.

Longitudinal data were drawn from the Morris Animal Foundation Golden Retriever Lifetime Study (n = 3,044). Stability was examined in terms of rank-order consistency (Pearson's r) and mean-level change (Wilcoxon signed-rank test). To test whether annual routines were associated with behavior in the following year, lagged linear mixed-effects models were fitted with adjustment for baseline problem level, age, and study year.

Behavioral outcomes showed moderate stability over time (r = 0.53-0.74), while also exhibiting small but statistically significant mean-level shifts: nonsocial fear, touch sensitivity, and composite fear- and stress-related scores tended to increase over time, whereas separation-related scores tended to decrease. Activity routines were also relatively stable, but more behaviorally plastic than the outcomes themselves. In the mixed-effects models, the strongest predictor of future behavior was the current level of the same outcome (β = 0.52-0.72, $p < 0.001$). Overall activity level nevertheless contributed a small but statistically significant additional signal, with higher activity predicting lower subsequent nonsocial fear and touch sensitivity. Predictive performance remained moderate on the validation set (R^2 = 0.32-0.49).

These findings indicate that fear- and anxiety-related outcomes in this cohort were moderately stable but not static, and that annual walking and activity routines contributed a small yet interpretable amount of additional information about later behavior. Overall activity level emerged as the most informative routine-related predictor. Everyday activity may therefore represent one modest but potentially modifiable component within the broader set of factors shaping dogs' emotional well-being.

Background

Fear- and anxiety-related behavior problems are common in companion dogs and are important from both welfare and clinical perspectives. Large-scale epidemiologic studies indicate that these traits are frequent in pet dogs. For example, in a study of 13,700 Finnish dogs, noise sensitivity was reported in 32% of dogs, fear in 29%, and aggression in 14%, whereas separation-related problems were less frequent under stricter case definitions (Salonen et al., 2020). More recently, a large U.S. analysis of behavioral data from 43,517 dogs enrolled in the Dog Aging Project reported that over 99% of dogs had at least one owner-rated behavior problem in the moderate-to-severe range, with separation-related and attachment behaviors receiving the highest proportion of moderate-to-severe ratings (Beaver, 2024).

These problems also matter in practical welfare contexts, including rehoming and relinquishment. According to ASPCA shelter statistics, approximately 5.8 million dogs and cats entered U.S. shelters in 2024, and 607,000 animals were euthanized, underscoring the scale of companion-animal welfare challenges in the United States (ASPCA, 2024).

Everyday walking and activity routines are often assumed to influence canine behavior. Physical exercise and structured activity are thought to help regulate arousal and reduce the expression of some undesirable behaviors. Consistent with this possibility, a questionnaire-based study comparing 94 sport dogs with 140 more sedentary dogs found lower expression of several undesirable behaviors in the more active group, including some stereotypic behaviors; dogs participating in agility also showed lower dog-directed aggression in that sample (Zilocchi et al., 2016). Similarly, in another questionnaire-based study, a daily walk lasting at least one hour was associated with more controlled behavior, including less leash pulling, jumping on people, and chasing cats (Sergi & Panizzolo, 2019).

Even so, the current evidence base remains limited in several important ways. Much of the available research linking activity and behavior in dogs is cross-sectional, questionnaire-based, or focused on selected subpopulations such as sport dogs. These designs are useful for identifying broad associations, but they do not clearly distinguish relatively stable individual differences from within-dog change over time. Yet this distinction is central for interpretation: if fear-related behavior is relatively stable, whereas walking and activity routines are more plastic, then the question of whether ordinary variation in annual walking and activity patterns predicts later behavioral change requires a genuinely longitudinal design.

Several findings support this contrast. In a large Dog Aging Project analysis, mean scores in the fear domain were broadly stable across years at the population level (Li et al., 2025). In the same direction, Tiira and Lohi (2014) reported good test-retest

reliability for a canine anxiety questionnaire, with fear-related responses toward people, dogs, and situations showing high consistency across an eight-month interval. By contrast, physical activity measures appear more flexible. In a large accelerometer-based population study, dog activity levels varied systematically with age, season, day of week, and living conditions (O'Rourke et al., 2025). Taken together, these findings suggest that fear-related behavior may form a relatively stable background, whereas activity and walking routines may be more sensitive to everyday variation. This makes it especially important to study both types of measures within the same dogs over time.

The Morris Animal Foundation Golden Retriever Lifetime Study (GRLS) provides a useful setting for this question. GRLS is a large prospective cohort of 3,044 U.S. Golden Retrievers enrolled between 6 months and 2 years of age and followed over time with repeated annual data collection on health, lifestyle, environment, and behavior (Labadie et al., 2022). Annual owner-reported measures of activity, walking routines, and behavior make it possible to construct within-dog longitudinal records and examine both year-to-year stability and year-to-year change.

Against this background, the present study uses longitudinal GRLS data to address two main questions. First, how stable are fear- and anxiety-related behavioral measures, as well as walking and activity routine measures, across years? Second, do annual walking and activity routines predict subsequent levels of these behavioral outcomes after accounting for prior problem level, age, and study year? Because the data consist of repeated annual observations from the same dogs, the analysis uses a longitudinal mixed-effects modeling framework. Such models are well suited to repeated-measures data because they account for non-independence of observations within dogs, unequal numbers of observations, and the unbalanced structure typical of longitudinal datasets (Muth et al., 2016) and have previously been applied in longitudinal studies of dog behavior based on repeated observations of the same animals (Goold & Newberry, 2017). In the present study, this framework makes it possible to model next-year behavior as a function of prior behavior, age, study year, and encoded walk/activity predictors, thereby shifting the focus from simple cross-sectional contrasts between more and less active dogs to directional within-dog associations over time. Sensitivity analysis and scenario analysis are then used to examine how strongly the fitted models respond to changes in routine-related predictors and how the model behaves under realistic activity profiles. The aim is to provide a cautious, data-driven, model-based assessment of whether ordinary variation in everyday activity and walking routines is associated with modest improvement or worsening in fear- and anxiety-related behavioral outcomes, while recognizing that causal inference remains limited in observational cohort data.

Methods

Data sources

This analysis used annual survey data from the Morris Animal Foundation Golden Retriever Lifetime Study. Three primary data sources were combined: (1) an activity and walking survey with self-reported information on general activity level, walking frequency, duration, pace, off-leash opportunities, and related exercise variables; (2) a behavior survey providing standardized scores for fear-, stress-, and separation-related behaviors; and (3) a dog profile file used to obtain dates of birth and enrollment and to derive age at each study year.

Analyses were restricted to dogs with sufficient repeated annual data to construct within-dog yearly records and, for the lagged models, at least one adjacent year pair linking year (t) to year (t+1).

Variable construction

Activity and walk predictors

Activity- and walk-related variables were encoded as numeric predictors suitable for longitudinal modeling. Categorical responses for general activity level, walking frequency, walking duration, walking pace, and fetch-game frequency were mapped to ordered integer scales, with higher values indicating higher intensity or frequency. Walking without a leash was represented as a binary indicator.

Off-leash activity was additionally summarized using a composite off-leash activity index. This index was based on the off-leash aerobic exercise block, including aerobic exercise frequency, duration, and pace. For dogs without off-leash access, these off-leash aerobic components were structurally set to zero. Remaining missing values in the off-leash aerobic variables were also filled with zero before standardization. Standardized values of off-leash aerobic frequency, duration, and pace were then averaged to form the composite index, and dogs with no off-leash walking were explicitly assigned an index value of 0.0.

Overall activity level was represented as an ordered numeric predictor based on the reported general activity category.

Behavior outcomes

Behavior outcomes were summarized using both individual subscales and composite measures. A fear composite, `fear_composite`, was defined as the mean of stranger-directed fear, dog-directed fear, and nonsocial fear. A stress-related composite, `stress_related_comp`, was defined as the mean of nonsocial fear, separation-related problems, and touch sensitivity. These composite measures, together with selected individual subscales, served as the main outcomes for longitudinal modeling. All outcomes were treated as continuous scales.

Yearly analysis table

Annual activity/walk and behavior records were merged by `subject_id` and `year_in_study`. Age in years at each study year was derived from the dog profile file and joined to the yearly table. The resulting yearly analysis table aligned each dog's walk and activity variables with fear- and stress-related behavior outcomes from the same study year and included age and study-year indicators. This merged dataset was used for descriptive summaries and for construction of the lagged dataset.

Descriptive and stability analysis

Descriptive statistics were used to summarize the distribution of key behavior outcomes and walk/activity predictors by study year, including means, standard deviations, medians, and sample sizes.

Year-to-year stability was then quantified for both outcomes and predictors using Pearson correlations between measurements in year (t) and year (t+1), together with summaries of simple change scores. These summaries included mean and median year-to-year change, mean absolute change, proportions of dogs whose scores increased, decreased, or remained unchanged, and the distribution of non-zero changes. These analyses were used to characterize whether the measures behaved more like relatively stable traits or more dynamic states and to inform interpretation of the lagged models.

Lagged dataset construction

To analyze prediction over time, a lagged dataset was constructed from consecutive year pairs for each dog. For each transition (t \rightarrow t+1), behavior outcomes at time (t+1) were matched with the same outcomes at time (t), together with walk and activity predictors, age, and study-wave indicators measured at time (t). Only adjacent study years (0→1 and 1→2) were retained in order to avoid nonconsecutive gaps. Observations with missing outcome or predictor values required for a given model were excluded through listwise deletion after structural coding of the off-leash variables. The resulting lagged dataset preserved within-dog temporal ordering and formed the basis of the longitudinal mixed-effects analyses.

Longitudinal mixed-effects models

Model formulation

The primary longitudinal model is shown in equation (1). For each outcome (Y), the lagged model for dog (i) from year (t) to year (t+1) was

$$Y_{i,t+1} = \beta_0 + \beta_1 Y_{i,t} + \beta_2^\top X_{i,t} + \beta_3 \,\mathrm{age}_{i,t} + \beta_4^\top Z_{i,t} + u_i + \epsilon_{i,t} \tag{1}$$

where (Y_{i,t+1}) is the outcome at time (t+1), (Y_{i,t}) is the same outcome at time (t), (X_{i,t}) is the vector of walk and activity predictors measured at time (t), (\mathrm{age} *{i,t}) is age in years at time (t), and (Z*{i,t}) represents study-wave indicators. The

walk/activity predictor vector included encoded overall activity level, walk frequency, the binary off-leash indicator, the composite off-leash activity index, and fetch-game frequency. The term (u_i \sim N(0,\sigma_u^2)) is a dog-level random intercept capturing between-dog variability, and (\epsilon_{i,t} \sim N(0,\sigma^2)) is the residual error.

A same-year model was also fitted to estimate cross-sectional associations between annual routines and concurrent outcomes, as shown in equation (2):

$$Y_{i,t} = \alpha_0 + \alpha_1^\top X_{i,t} + \alpha_2 \,\mathrm{age}_{i,t} + \alpha_3^\top Z_{i,t} + u_i + \epsilon_{i,t} \tag{2}$$

where the notation is analogous to equation (1), except that the outcome is modeled within the same study year rather than across adjacent years.

All mixed models were fit in Python using `statsmodels` (`MixedLM`) with dog-level random intercepts and maximum-likelihood estimation. For lagged models, several optimizers were tried in sequence, and the first converged solution was retained.

Train-test split

To assess out-of-sample performance, dogs were randomly split into a calibration (training) set and a validation (test) set at the dog level, ensuring that all repeated observations for a given dog remained in the same subset. Mixed models were fit on the training subset of the lagged dataset and evaluated on the held-out test subset using (R^2), root mean squared error (RMSE), and Nash-Sutcliffe efficiency for each outcome.

Sensitivity analysis

A prediction-based single-parameter sensitivity analysis was conducted for all walk- and activity-related predictors included in the main lagged models. In this step, one predictor was varied at a time while all remaining predictors were held constant at typical values. For ordinal predictors, sensitivity was evaluated across the observed minimum-to-maximum range. For the continuous composite off-leash activity index, sensitivity was evaluated from the 25th to the 75th percentile.

For each outcome, the analysis estimated how much the model-predicted next-year value changed across the selected contrast for each predictor. Sensitivity was summarized using absolute change in the predicted outcome, relative percentage change, and change per unit of predictor contrast. This analysis was used to compare the relative directional influence of the walk- and activity-related predictors within the fitted longitudinal models.

Scenario analysis

To complement the single-parameter sensitivity analysis, a scenario analysis was performed using the fitted lagged mixed-effects models. A small set of plausible routine

profiles was defined by combining low, medium, and high overall activity levels with lower versus higher values of the composite off-leash activity index, while baseline outcome, age, and the remaining predictors were held constant at typical values.

Model-predicted next-year values were then generated for two illustrative outcomes, the stress-related composite and nonsocial fear. This profile-based analysis provided a more interpretable summary of how the fitted models behaved under contrasting but realistic activity and walking routines. The purpose of this step was illustrative rather than causal.

Results

This section first summarizes the descriptive patterns and year-to-year stability of the behavioral outcomes and walking/activity measures. It then presents the same-year and lagged mixed-effects models, followed by model validation, prediction-based sensitivity analysis, and scenario analysis.

Data assembly and preprocessing checks

Step 1. Load and inspect the source data

At this step, we load the annual activity and behavior datasets and inspect their basic structure.

The goal is to verify the available variables, the number of observations, and the identifiers needed for yearly and longitudinal analyses.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

# ----------------------------------------------------------------------
# Step 1. Load and inspect activity, behavior, and dog profile data
# ----------------------------------------------------------------------

PATH = Path("./")

activity_file = PATH / "activity_details_through_SY2.csv"
behavior_file = PATH / "beh_problem_scores.csv"
profile_file = PATH / "dog_profile.csv"

activity = pd.read_csv(activity_file)
behavior = pd.read_csv(behavior_file)
dog_profile = pd.read_csv(profile_file)

print("Activity shape:", activity.shape)
```

```
print("Behavior shape:", behavior.shape)
print("Dog profile shape:", dog_profile.shape)

print("\nActivity columns:")
print(activity.columns.tolist())

print("\nBehavior columns:")
print(behavior.columns.tolist())

print("\nDog profile columns:")
print(dog_profile.columns.tolist())

display(activity.head())
display(behavior.head())
display(dog_profile.head())

# ---------------------------------------------------------------------
# Build an age layer from birth_date and enrolled_date
# ---------------------------------------------------------------------

profile_age = dog_profile[["subject_id", "birth_date", "enrolled_date"]].cop

profile_age["birth_date"] = pd.to_datetime(profile_age["birth_date"], format
profile_age["enrolled_date"] = pd.to_datetime(profile_age["enrolled_date"],

profile_age["baseline_age_years"] = (
    (profile_age["enrolled_date"] - profile_age["birth_date"]).dt.days / 365
)

# Create the subject-year key space from activity and behavior
analysis_keys = (
    activity[["subject_id", "year_in_study"]]
    .drop_duplicates()
    .merge(
        behavior[["subject_id", "year_in_study"]].drop_duplicates(),
        on=["subject_id", "year_in_study"],
        how="inner"
    )
    .sort_values(["subject_id", "year_in_study"])
    .reset_index(drop=True)
)

# Expand baseline age to each study year
age_df = analysis_keys.merge(
    profile_age[["subject_id", "baseline_age_years"]],
    on="subject_id",
    how="left"
)

age_df["age_years"] = age_df["baseline_age_years"] + age_df["year_in_study"]
age_df = age_df[["subject_id", "year_in_study", "age_years"]].copy()

print("\nAge layer shape:", age_df.shape)

print("\nMissingness in age layer:")
print(age_df.isna().mean())
```

```
display(age_df.head())
```

```
Activity shape: (8354, 16)
Behavior shape: (21184, 12)
Dog profile shape: (3044, 11)

Activity columns:
['subject_id', 'year_in_study', 'record_date', 'activity_level', 'walk_frequ
ency', 'walk_duration', 'walk_pace', 'walk_reason_bathroom', 'walk_reason_ge
neral_enjoyment', 'walk_reason_training', 'walk_reason_other_specify', 'walk
_without_leash', 'aerobic_duration', 'aerobic_pace', 'aerobic_frequency', 'f
etch_games_frequency']

Behavior columns:
['subject_id', 'year_in_study', 'record_date', 'score_stranger_directed_aggr
ession', 'score_owner_directed_aggression', 'score_dog_directed_aggression',
'score_dog_directed_fear', 'score_stranger_directed_fear', 'score_nonsocial_
fear', 'score_separation_related_problems', 'score_touch_sensitivity', 'scor
e_excitability']

Dog profile columns:
['subject_id', 'enrolled_date', 'sex_status', 'spay_neuter_date', 'study_sta
tus', 'birth_date', 'withdrawn_date', 'inactive_date', 'death_date', 'is_eut
hanized', 'coat_color']
```

	subject_id	year_in_study	record_date	activity_level	walk_frequency	walk_dur
0	grls007MPNZZ	0	2014-05	Very active	More than once a day	30-60 mi
1	grls007MPNZZ	1	2015-05	Very active	More than once a day	10-30 mi
2	grls007MPNZZ	2	2016-05	Moderate	More than once a day	30-60 mi
3	grls00NB23WW	0	2014-01	Very active	Less than once/month	10-30 mi
4	grls00NB23WW	1	2015-01	Very active	Less than once/month	30-60 mi

	subject_id	year_in_study	record_date	score_stranger_directed_aggression	scor
0	grls007MPNZZ	0	2014-05	0.0	
1	grls007MPNZZ	1	2015-05	0.0	
2	grls007MPNZZ	2	2016-05	0.2	
3	grls007MPNZZ	3	2017-05	0.0	
4	grls007MPNZZ	4	2018-05	0.2	

	subject_id	enrolled_date	sex_status	spay_neuter_date	study_status	birth_date
0	grlsOFX17FUU	2012-09	Neutered Male	2011-08	Enrolled Deceased	2011-02
1	grls3SN3WPJJ	2012-10	Intact Male	NaN	Enrolled Deceased	2011-04
2	grls3XQD0E33	2012-10	Intact Male	NaN	Withdrawn	2010-12
3	grlsJ4RY5O33	2012-11	Spayed Female	2011-08	Enrolled Deceased	2011-03
4	grlsGQEN0GEE	2012-10	Spayed Female	2012-11	Enrolled Deceased	2011-04

```
Age layer shape: (8352, 3)

Missingness in age layer:
subject_id       0.0
year_in_study    0.0
age_years        0.0
dtype: float64
```

	subject_id	year_in_study	age_years
0	grls007MPNZZ	0	1.251198
1	grls007MPNZZ	1	2.251198
2	grls007MPNZZ	2	3.251198
3	grls00NB23WW	0	2.001369
4	grls00NB23WW	1	3.001369

In [2]:

```python
# ----------------------------------------------------------------------
# Quick duplicate checks
# ----------------------------------------------------------------------

activity_dups = activity.duplicated(subset=["subject_id", "year_in_study"]).
behavior_dups = behavior.duplicated(subset=["subject_id", "year_in_study"]).
age_dups = age_df.duplicated(subset=["subject_id", "year_in_study"]).sum()

print("Duplicate subject-year rows in activity:", activity_dups)
print("Duplicate subject-year rows in behavior:", behavior_dups)
print("Duplicate subject-year rows in age layer:", age_dups)
```

```
Duplicate subject-year rows in activity: 0
Duplicate subject-year rows in behavior: 0
Duplicate subject-year rows in age layer: 0
```

The activity, behavior, and dog profile datasets were loaded and inspected.
At this stage, the goal is to confirm file structure, key variables, and readiness for merging by `subject_id` and `year_in_study`.

An age layer was also constructed from `birth_date` and `enrolled_date`. Because the profile dates are recorded at the month level, baseline age was approximated from the difference between enrollment month and birth month, and yearly age was then defined as:

`age_years = baseline_age_years + year_in_study`

This age layer will be merged into the yearly analysis table in the next step.

Step 2. Encode activity variables and build core derived predictors

In this step, ordinal activity variables are converted into numeric scores. Structural zeros are assigned where a type of activity was not present:

- no leash walks -> leash duration and leash pace set to zero
- no off-leash access -> off-leash exercise variables set to zero

This step also creates the core off-leash dose index used in later models.

In [3]:
```
# Encode activity variables and create core derived predictors

freq_map = {
    "Never": 0,
    "Less than once/month": 1,
    "1-3 times/month": 2,
    "Less than once/week": 3,
    "Once or twice/week": 4,
    "2-6 times/week": 5,
    "Once/day": 6,
    "More than once a day": 7,
}

dur_map = {
    "Less than 10 minutes": 1,
    "10-30 minutes": 2,
    "30-60 minutes": 3,
    "Greater than 60 minutes": 4,
}

pace_map = {
    "Slow": 1,
    "Average": 2,
    "Brisk": 3,
    "Jog": 4,
    "Run": 5,
}

aer_pace_map = {
    "Bursts of exercise less than 30% of the time": 1,
    "Engages in moderate exercise (greater than 30% of the time but less tha
```

```
    "Engages in strenuous exercise most of the time (greater than 60%)": 3,
}

activity_level_map = {
    "Little": 0,
    "Moderate": 1,
    "Very active": 2,
}

fetch_map = {
    "Never": 0,
    "Occasionally": 1,
    "Frequently": 2,
}

act = activity.copy()

# Basic encoded variables
act["activity_level_num"] = act["activity_level"].map(activity_level_map)
act["walk_frequency_num"] = act["walk_frequency"].map(freq_map)
act["walk_duration_num"] = act["walk_duration"].map(dur_map)
act["walk_pace_num"] = act["walk_pace"].map(pace_map)
act["walk_without_leash_num"] = pd.to_numeric(act["walk_without_leash"], err
act["fetch_games_frequency_num"] = act["fetch_games_frequency"].map(fetch_ma

act["aerobic_frequency_num"] = act["aerobic_frequency"].map(freq_map)
act["aerobic_duration_num"] = act["aerobic_duration"].map(dur_map)
act["aerobic_pace_num"] = act["aerobic_pace"].map(aer_pace_map)

# Structural zeros for leash block if no leash walks
no_leash = act["walk_frequency_num"] == 0
act.loc[no_leash, "walk_duration_num"] = 0
act.loc[no_leash, "walk_pace_num"] = 0

# Structural zeros for off-leash block if no off-leash access
no_offleash = act["walk_without_leash_num"] == 0
act.loc[no_offleash, "aerobic_frequency_num"] = 0
act.loc[no_offleash, "aerobic_duration_num"] = 0
act.loc[no_offleash, "aerobic_pace_num"] = 0

# Fill any remaining off-leash missing values with zero for the dose index
act["aerobic_frequency_filled"] = act["aerobic_frequency_num"].fillna(0)
act["aerobic_duration_filled"] = act["aerobic_duration_num"].fillna(0)
act["aerobic_pace_filled"] = act["aerobic_pace_num"].fillna(0)

# Standardized off-leash dose index
for c in ["aerobic_frequency_filled", "aerobic_duration_filled", "aerobic_pa
    std = act[c].std()
    if pd.isna(std) or std == 0:
        act[f"{c}_z"] = 0.0
    else:
        act[f"{c}_z"] = (act[c] - act[c].mean()) / std

act["offleash_dose_index_filled"] = act[
    ["aerobic_frequency_filled_z", "aerobic_duration_filled_z", "aerobic_pac
].mean(axis=1)
```

```python
# Keep only variables needed downstream
act_keep = [
    "subject_id",
    "year_in_study",
    "activity_level_num",
    "walk_frequency_num",
    "walk_duration_num",
    "walk_pace_num",
    "walk_without_leash_num",
    "fetch_games_frequency_num",
    "aerobic_frequency_num",
    "aerobic_duration_num",
    "aerobic_pace_num",
    "offleash_dose_index_filled",
]

act = act[act_keep].copy()

print("Encoded activity shape:", act.shape)
print("\nMissingness in encoded activity variables:")
print(act.isna().mean().sort_values(ascending=False))

display(act.head(10))
```

```
Encoded activity shape: (8354, 12)

Missingness in encoded activity variables:
activity_level_num            0.000359
aerobic_pace_num              0.000239
aerobic_duration_num          0.000239
walk_duration_num             0.000120
aerobic_frequency_num         0.000120
walk_pace_num                 0.000120
subject_id                    0.000000
year_in_study                 0.000000
walk_frequency_num            0.000000
walk_without_leash_num        0.000000
fetch_games_frequency_num     0.000000
offleash_dose_index_filled    0.000000
dtype: float64
```

	subject_id	year_in_study	activity_level_num	walk_frequency_num	walk_duratic
0	grls007MPNZZ	0	2.0	7	
1	grls007MPNZZ	1	2.0	7	
2	grls007MPNZZ	2	1.0	7	
3	grls00NB23WW	0	2.0	1	
4	grls00NB23WW	1	2.0	1	
5	grls00NB23WW	2	2.0	1	
6	grls01C2L8RR	0	1.0	4	
7	grls01C2L8RR	1	1.0	3	
8	grls01C2L8RR	2	2.0	3	
9	grls01LNL888	0	2.0	0	

The activity variables were encoded successfully.

At this stage, leash-related and off-leash-related variables are represented numerically, and structural zeros were assigned where the corresponding type of activity was absent.

The off-leash dose index summarizes the relative intensity of off-leash activity using standardized frequency, duration, and pace components.

A small adjustment is applied to the off-leash dose index so that dogs without off-leash access receive a dose value of exactly zero. This improves interpretability in later descriptive and longitudinal models.

Step 3. Build the yearly analysis table

At this step, we merge the yearly activity and behavior data by dog and study year.

We also construct the composite outcomes used in the main analyses. The resulting yearly table serves as the core dataset for descriptive, stability, and longitudinal modeling steps.

In [4]:
```
# Step 3. Adjust off-leash dose coding

act["offleash_dose_index_filled"] = act["offleash_dose_index_filled"].copy()
act.loc[act["walk_without_leash_num"] == 0, "offleash_dose_index_filled"] =

print("Updated off-leash dose summary:")
print(act["offleash_dose_index_filled"].describe())

display(
    act[
        [
```

```
            "subject_id",
            "year_in_study",
            "walk_without_leash_num",
            "aerobic_frequency_num",
            "aerobic_duration_num",
            "aerobic_pace_num",
            "offleash_dose_index_filled",
        ]
    ].head(10)
)
```

```
Updated off-leash dose summary:
count    8354.000000
mean        0.464399
std         0.538566
min        -0.892681
25%         0.000000
50%         0.000000
75%         0.969761
max         1.781896
Name: offleash_dose_index_filled, dtype: float64
```

	subject_id	year_in_study	walk_without_leash_num	aerobic_frequency_num	aer
0	grls007MPNZZ	0	0	0.0	
1	grls007MPNZZ	1	0	0.0	
2	grls007MPNZZ	2	1	6.0	
3	grls00NB23WW	0	0	0.0	
4	grls00NB23WW	1	0	0.0	
5	grls00NB23WW	2	0	0.0	
6	grls01C2L8RR	0	0	0.0	
7	grls01C2L8RR	1	0	0.0	
8	grls01C2L8RR	2	0	0.0	
9	grls01LNL888	0	1	7.0	

Step 4. Merge yearly behavior and activity data

In [5]:
```
# Step 4. Merge yearly behavior, activity, and age data

yearly = (
    behavior
    .merge(act, on=["subject_id", "year_in_study"], how="inner")
    .merge(age_df, on=["subject_id", "year_in_study"], how="left")
)

# Build outcome composites
fear_cols = [
    "score_dog_directed_fear",
```

```
    "score_stranger_directed_fear",
    "score_nonsocial_fear",
]

yearly["fear_composite"] = yearly[fear_cols].mean(axis=1)

yearly["stress_related_comp"] = yearly[
    [
        "score_nonsocial_fear",
        "score_separation_related_problems",
        "score_touch_sensitivity",
    ]
].mean(axis=1)

# Build routine type
leash_present = yearly["walk_frequency_num"] > 0
offleash_present = yearly["walk_without_leash_num"] == 1

yearly["routine_type"] = np.select(
    [
        (~leash_present) & (~offleash_present),
        (leash_present) & (~offleash_present),
        (~leash_present) & (offleash_present),
        (leash_present) & (offleash_present),
    ],
    [
        "none",
        "leash_only",
        "offleash_only",
        "both",
    ],
    default="missing"
)

# Save an analysis-ready alias for later steps
yearly_analysis_df = yearly.copy()

# Quick QC
print("Yearly merged shape:", yearly.shape)
print("Unique dogs:", yearly["subject_id"].nunique())
print("Duplicate subject-year rows:", yearly.duplicated(subset=["subject_id"

print("\nMissingness in key outcomes:")
print(
    yearly[
        [
            "fear_composite",
            "score_nonsocial_fear",
            "score_separation_related_problems",
            "score_touch_sensitivity",
            "stress_related_comp",
        ]
    ].isna().mean()
)

print("\nMissingness in key predictors and age:")
```

```python
print(
    yearly[
        [
            "activity_level_num",
            "walk_frequency_num",
            "walk_without_leash_num",
            "offleash_dose_index_filled",
            "fetch_games_frequency_num",
            "age_years",
        ]
    ].isna().mean()
)

print("\nRoutine type distribution:")
print(yearly["routine_type"].value_counts(normalize=True))

display(
    yearly[
        [
            "subject_id",
            "year_in_study",
            "age_years",
            "fear_composite",
            "score_nonsocial_fear",
            "score_separation_related_problems",
            "score_touch_sensitivity",
            "stress_related_comp",
            "activity_level_num",
            "walk_frequency_num",
            "walk_without_leash_num",
            "offleash_dose_index_filled",
            "fetch_games_frequency_num",
            "routine_type",
        ]
    ].head(10)
)
```

```
Yearly merged shape: (8352, 26)
Unique dogs: 3044
Duplicate subject-year rows: 0

Missingness in key outcomes:
fear_composite                      0.002993
score_nonsocial_fear                0.010297
score_separation_related_problems   0.019756
score_touch_sensitivity             0.028496
stress_related_comp                 0.000479
dtype: float64

Missingness in key predictors and age:
activity_level_num            0.000359
walk_frequency_num            0.000000
walk_without_leash_num        0.000000
offleash_dose_index_filled    0.000000
fetch_games_frequency_num     0.000000
age_years                     0.000000
dtype: float64

Routine type distribution:
routine_type
leash_only       0.506346
both             0.447797
offleash_only    0.031968
none             0.013889
Name: proportion, dtype: float64
```

	subject_id	year_in_study	age_years	fear_composite	score_nonsocial_fear	sco
0	grls007MPNZZ	0	1.251198	0.860000	1.83	
1	grls007MPNZZ	1	2.251198	1.000000	2.00	
2	grls007MPNZZ	2	3.251198	0.723333	1.50	
3	grls00NB23WW	0	2.001369	0.000000	NaN	
4	grls00NB23WW	1	3.001369	0.056667	0.17	
5	grls00NB23WW	2	4.001369	0.330000	0.33	
6	grls01C2L8RR	0	0.495551	0.526667	0.83	
7	grls01C2L8RR	1	1.495551	0.333333	1.00	
8	grls01C2L8RR	2	2.495551	0.416667	1.00	
9	grls01LNL888	0	1.166324	0.266667	0.80	

The yearly analysis table now combines behavior outcomes, encoded walk/activity predictors, and dog age for each available `subject_id` by `year_in_study`.

At this stage, the main yearly variables used in later analyses are available in one table: key outcomes, composite scores, routine-related predictors, `age_years`, and `routine_type`.

This merged table serves as the core analysis layer for descriptive summaries, same-year models, and construction of the lagged dataset.

In [6]:
```
# Quick yearly structure check

print("Years available:", sorted(yearly_analysis_df["year_in_study"].dropna(

print("\nRows by study year:")
display(yearly_analysis_df["year_in_study"].value_counts().sort_index())

print("\nUnique dogs by study year:")
display(
    yearly_analysis_df.groupby("year_in_study")["subject_id"].nunique()
)
```

```
Years available: [np.int64(0), np.int64(1), np.int64(2)]

Rows by study year:
year_in_study
0    3044
1    2705
2    2603
Name: count, dtype: int64
Unique dogs by study year:
year_in_study
0    3044
1    2705
2    2603
Name: subject_id, dtype: int64
```

Step 5. Descriptive statistics for outcomes

This section summarizes the main behavior outcomes across study years.

The goal is to describe:

- central tendency and spread for each outcome
- whether group-level distributions appear broadly similar or shifted across years
- how much year-to-year movement is visible in the key behavior measures

Table 1. Descriptive statistics for the main behavioral outcomes across study years.

In [7]:
```
# Step 5. Descriptive statistics for outcomes

outcome_vars = [
    "score_nonsocial_fear",
    "score_separation_related_problems",
    "score_touch_sensitivity",
    "stress_related_comp",
    "fear_composite",
]
```

```
outcome_desc = (
    yearly_analysis_df
    .groupby("year_in_study")[outcome_vars]
    .agg(["mean", "std", "median", "count"])
    .round(3)
)

# Flatten the column index to make the table easier to read
outcome_desc.columns = [
    f"{var}_{stat}" for var, stat in outcome_desc.columns
]

display(outcome_desc)
```

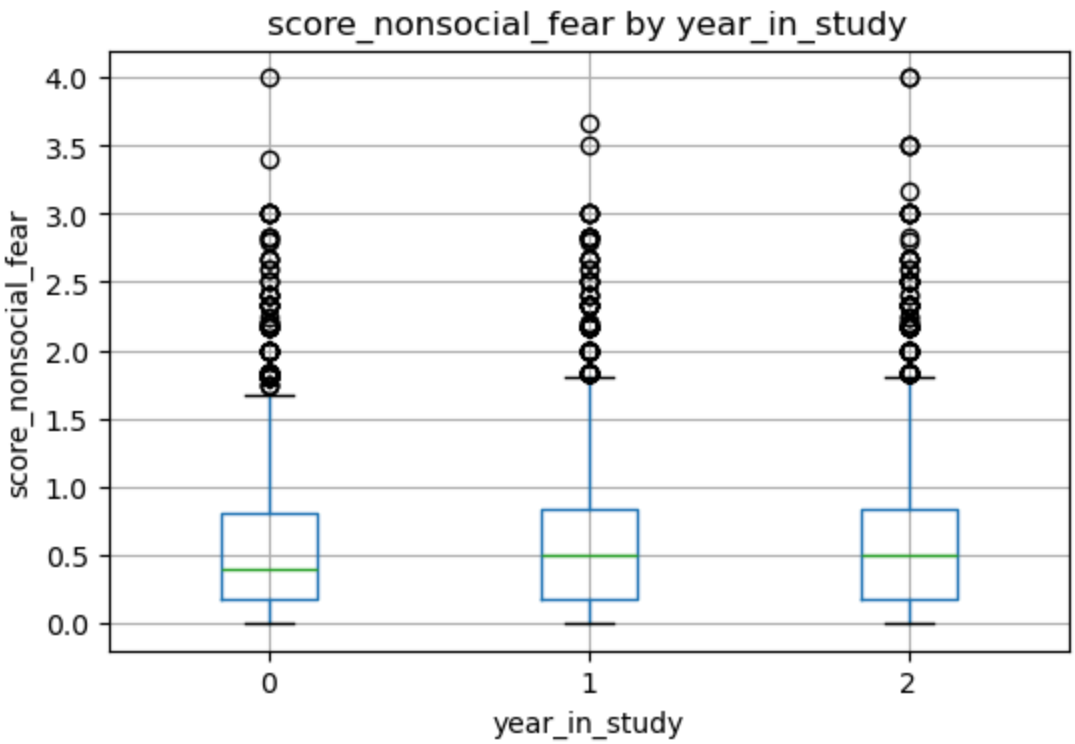
	score_nonsocial_fear_mean	score_nonsocial_fear_std	score_nonsocial_fe
year_in_study			
0	0.528	0.519	
1	0.574	0.564	
2	0.610	0.593	

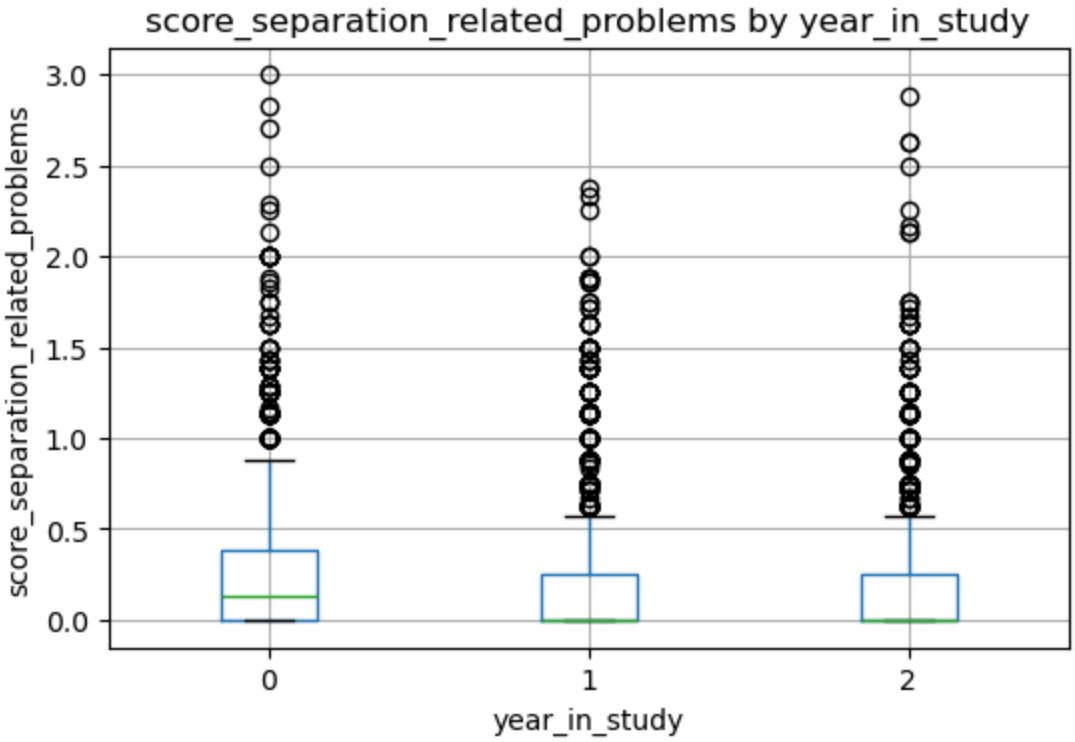
The table above provides a year-by-year overview of the main fear- and stress-related outcomes before longitudinal modeling.

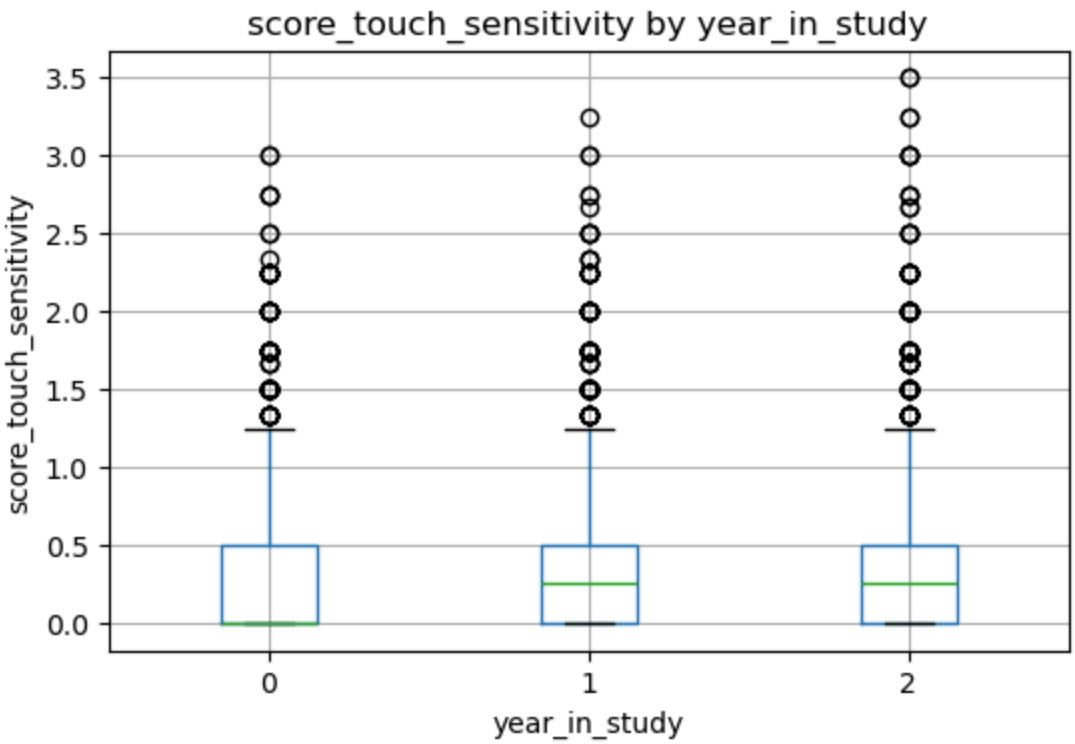
At this stage, the purpose is descriptive: to summarize central tendency, dispersion, and broad directional shifts across study years, without yet adjusting for age, repeated measures, or routine-related predictors.

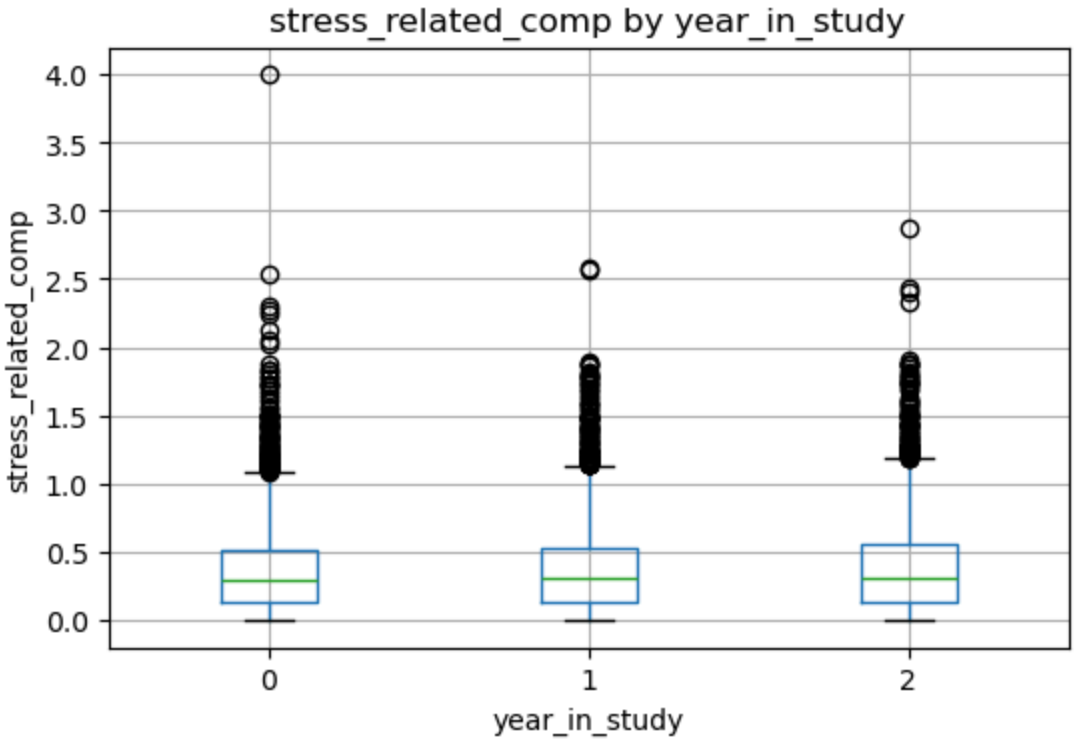
In [8]:
```
# Boxplots by year for outcomes

for var in outcome_vars:
    ax = yearly_analysis_df.boxplot(column=var, by="year_in_study", figsize=
    ax.set_title(f"{var} by year_in_study")
    plt.suptitle("")
    ax.set_xlabel("year_in_study")
    ax.set_ylabel(var)
    plt.show()
```


score_nonsocial_fear by year_in_study
score_nonsocial_fear
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
1
2
year_in_study


score_separation_related_problems by year_in_study
score_separation_related_problems
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
1
2
year_in_study


score_touch_sensitivity by year_in_study
score_touch_sensitivity
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
1
2
year_in_study


stress_related_comp by year_in_study
stress_related_comp
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
1
2
year_in_study

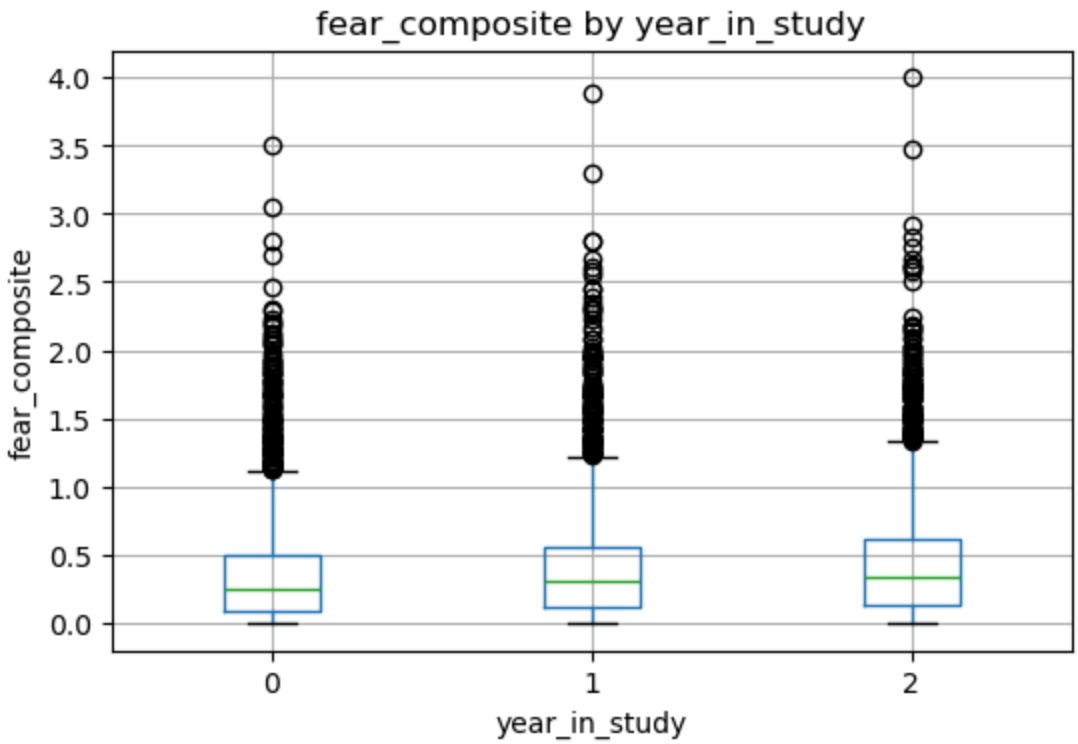


Figure 1. Distribution of the main behavioral outcomes across study years. Boxplots are shown separately for each outcome.

In [9]:
```
# Build consecutive year pairs for outcome stability plots

outcome_base = yearly_analysis_df[
    ["subject_id", "year_in_study"] + outcome_vars
].copy()

outcome_next = outcome_base.copy()
outcome_next["year_in_study"] = outcome_next["year_in_study"] - 1
outcome_next = outcome_next.rename(
    columns={v: f"{v}_next" for v in outcome_vars}
)

outcome_pairs = outcome_base.merge(
    outcome_next,
    on=["subject_id", "year_in_study"],
    how="inner"
)

outcome_pairs = outcome_pairs[outcome_pairs["year_in_study"].isin([0, 1])].c

for v in outcome_vars:
    outcome_pairs[f"delta_{v}"] = outcome_pairs[f"{v}_next"] - outcome_pairs

print("Outcome consecutive pairs shape:", outcome_pairs.shape)
print("Unique dogs in outcome pairs:", outcome_pairs["subject_id"].nunique()

display(outcome_pairs.head())
```

```
Outcome consecutive pairs shape: (5202, 17)
Unique dogs in outcome pairs: 2705
```

	subject_id	year_in_study	score_nonsocial_fear	score_separation_related_proble
0	grls007MPNZZ	0	1.83	(
1	grls007MPNZZ	1	2.00	(
2	grls00NB23WW	0	NaN	2
3	grls00NB23WW	1	0.17	2
4	grls01C2L8RR	0	0.83	(

In [10]:

```python
# Histograms of year-to-year change in outcomes with simple stability labels

thresholds = {
    "score_nonsocial_fear": 0.25,
    "score_separation_related_problems": 0.25,
    "score_touch_sensitivity": 0.25,
    "stress_related_comp": 0.20,
    "fear_composite": 0.20,
}

fig, axes = plt.subplots(2, 3, figsize=(15, 9))
axes = axes.flatten()

for i, var in enumerate(outcome_vars):
    data = outcome_pairs[f"delta_{var}"].dropna()
    thresh = thresholds[var]

    n = len(data)
    stable = (data.abs() <= thresh).sum() / n * 100 if n > 0 else np.nan
    decreased = (data < -thresh).sum() / n * 100 if n > 0 else np.nan
    increased = (data > thresh).sum() / n * 100 if n > 0 else np.nan

    axes[i].hist(data, bins=30)
    axes[i].axvline(0, color="black", linestyle="--", alpha=0.7)
    axes[i].set_title(var)
    axes[i].set_xlabel("Year-to-year change")
    axes[i].set_ylabel("Count")

    stats_text = (
        f"Stable: {stable:.1f}%\n"
        f"Decreased: {decreased:.1f}%\n"
        f"Increased: {increased:.1f}%"
    )
    axes[i].text(
        0.97, 0.95, stats_text,
        transform=axes[i].transAxes,
        ha="right", va="top",
        bbox=dict(boxstyle="round", facecolor="white", alpha=0.8)
    )

# Remove empty subplot
if len(outcome_vars) < len(axes):
```

```
    for j in range(len(outcome_vars), len(axes)):
        fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```

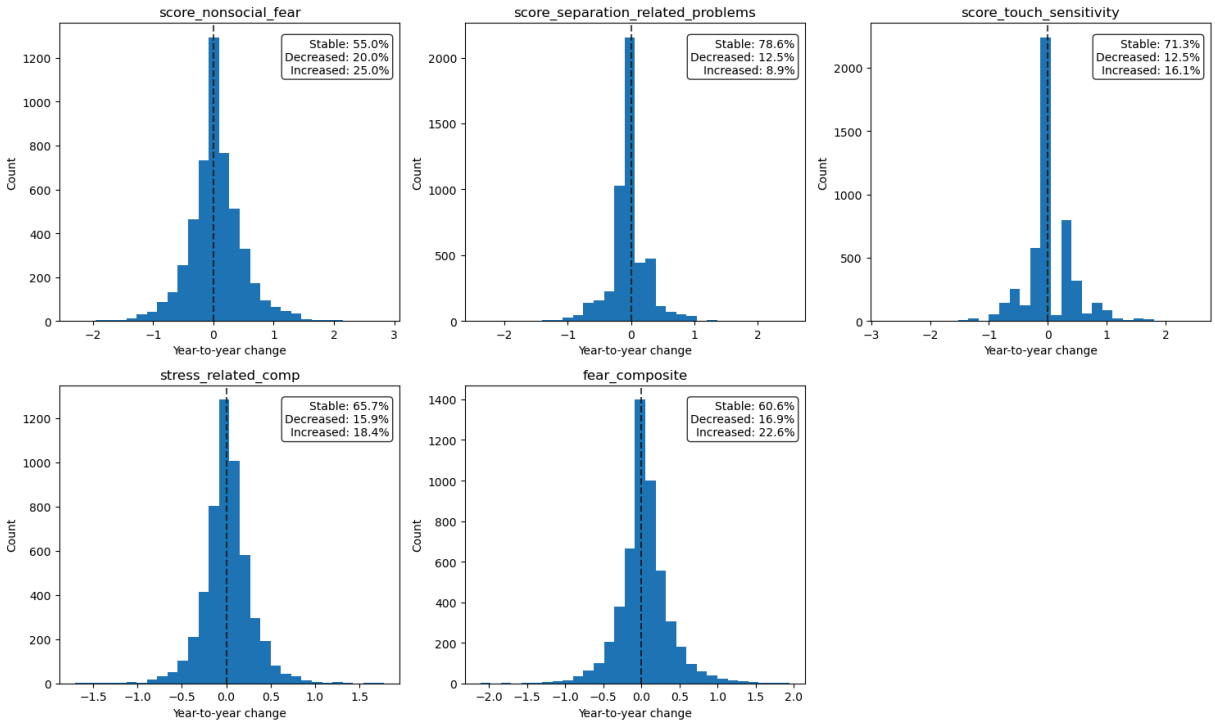


Figure 2. Distributions of year-to-year changes in the main behavioral outcomes across adjacent annual transitions.

Step 6. Descriptive statistics for predictors

This section summarizes the main walk and activity predictors across study years.

The goal is to describe:

- whether core walk/activity variables show population-level shifts over time
- whether dogs appear to maintain relatively stable routine profiles
- how much year-to-year movement is visible in the main predictors

Table 2. Descriptive statistics for the main walking and activity predictors across study years, together with routine-type proportions.

In [11]:
```
# Step 6. Descriptive statistics for predictors

predictor_vars = [
    "activity_level_num",
    "walk_frequency_num",
    "walk_without_leash_num",
    "offleash_dose_index_filled",
    "fetch_games_frequency_num",
```

```
]

predictor_summary = (
    yearly_analysis_df
    .groupby("year_in_study")[predictor_vars]
    .agg(["mean", "std", "median", "count"])
    .round(3)
)

# Flatten the column index to make the table easier to read
predictor_summary.columns = [
    f"{var}_{stat}" for var, stat in predictor_summary.columns
]

print("Predictor summary by year:")
display(predictor_summary)

print("\nRoutine type proportions by year:")
routine_by_year = pd.crosstab(
    yearly_analysis_df["year_in_study"],
    yearly_analysis_df["routine_type"],
    normalize="index"
).round(3)

display(routine_by_year)
```

```
Predictor summary by year:
```

year_in_study	activity_level_num_mean	activity_level_num_std	activity_level_num_med
0	1.500	0.518	
1	1.413	0.543	
2	1.299	0.527	

```
Routine type proportions by year:
```

routine_type	both	leash_only	none	offleash_only
year_in_study				
0	0.431	0.528	0.015	0.025
1	0.466	0.487	0.013	0.033
2	0.448	0.501	0.013	0.038

The table above summarizes the main walk/activity predictors across study years.

At this stage, the purpose is descriptive: to show central tendency, dispersion, and broad shifts in routine-related variables before formal stability analysis or mixed modeling.

In [12]:
```
# Routine type proportions by year
```

```
routine_by_year.plot(kind="bar", stacked=True, figsize=(8, 5))
plt.title("Routine type proportions by year")
plt.xlabel("Year in study")
plt.ylabel("Proportion")
plt.legend(title="Routine type", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
plt.show()
```

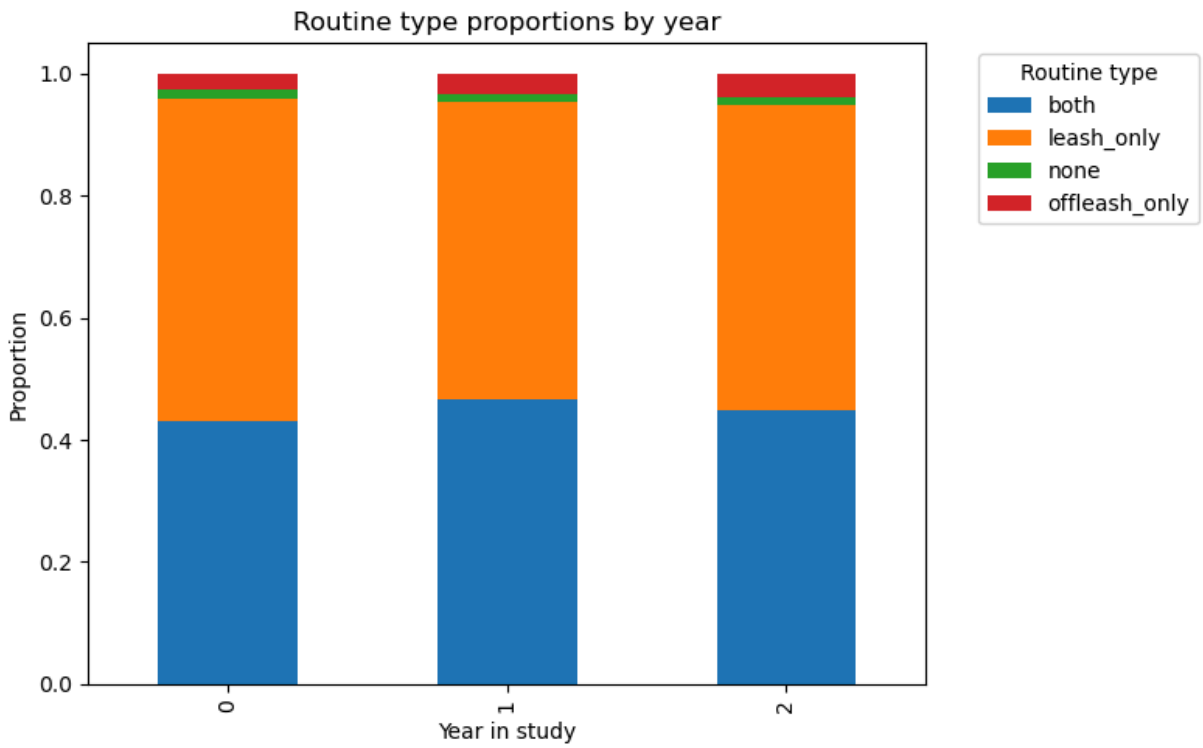


Figure 3. Proportions of routine types across study years.

In [13]:
```
# Build consecutive year pairs for predictor change plots

predictor_base = yearly_analysis_df[
    ["subject_id", "year_in_study"] + predictor_vars
].copy()

predictor_next = predictor_base.copy()
predictor_next["year_in_study"] = predictor_next["year_in_study"] - 1
predictor_next = predictor_next.rename(
    columns={v: f"{v}_next" for v in predictor_vars}
)

predictor_pairs = predictor_base.merge(
    predictor_next,
    on=["subject_id", "year_in_study"],
    how="inner"
)

predictor_pairs = predictor_pairs[predictor_pairs["year_in_study"].isin([0,

for v in predictor_vars:
    predictor_pairs[f"delta_{v}"] = predictor_pairs[f"{v}_next"] - predictor
```

```
print("Predictor consecutive pairs shape:", predictor_pairs.shape)
print("Unique dogs in predictor pairs:", predictor_pairs["subject_id"].nunic

display(predictor_pairs.head())
```

```
Predictor consecutive pairs shape: (5202, 17)
Unique dogs in predictor pairs: 2705
```

	subject_id	year_in_study	activity_level_num	walk_frequency_num	walk_withou
0	grls007MPNZZ	0	2.0	7	
1	grls007MPNZZ	1	2.0	7	
2	grls00NB23WW	0	2.0	1	
3	grls00NB23WW	1	2.0	1	
4	grls01C2L8RR	0	1.0	4	

In [14]:

```
# Histograms of year-to-year change in predictors with simple stability labe

thresholds_x = {
    "activity_level_num": 0.5,
    "walk_frequency_num": 0.5,
    "walk_without_leash_num": 0.5,
    "offleash_dose_index_filled": 0.5,
    "fetch_games_frequency_num": 0.5,
}

fig, axes = plt.subplots(2, 3, figsize=(15, 9))
axes = axes.flatten()

for i, var in enumerate(predictor_vars):
    data = predictor_pairs[f"delta_{var}"].dropna()
    thresh = thresholds_x[var]

    n = len(data)
    stable = (data.abs() <= thresh).sum() / n * 100 if n > 0 else np.nan
    decreased = (data < -thresh).sum() / n * 100 if n > 0 else np.nan
    increased = (data > thresh).sum() / n * 100 if n > 0 else np.nan

    axes[i].hist(data, bins=30)
    axes[i].axvline(0, color="black", linestyle="--", alpha=0.7)
    axes[i].set_title(var)
    axes[i].set_xlabel("Year-to-year change")
    axes[i].set_ylabel("Count")

    stats_text = (
        f"Stable: {stable:.1f}%\n"
        f"Decreased: {decreased:.1f}%\n"
        f"Increased: {increased:.1f}%"
    )

    axes[i].text(
        0.97,
```

```
        0.95,
        stats_text,
        transform=axes[i].transAxes,
        ha="right",
        va="top",
        bbox=dict(boxstyle="round", facecolor="white", alpha=0.8),
    )

# Hide the unused last subplot
axes[-1].axis("off")

plt.tight_layout()
plt.show()
```

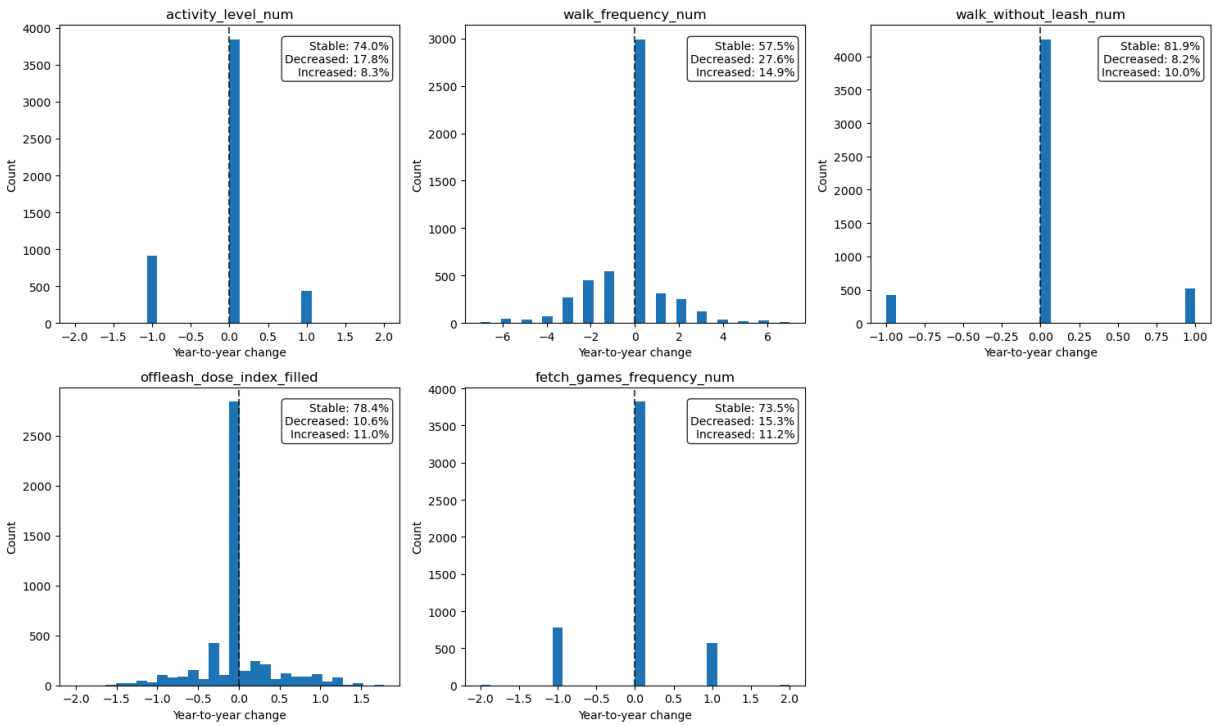


Figure 4. Distributions of year-to-year changes in the main walking and activity predictors across adjacent annual transitions.

Step 7. Stability analysis for outcomes

This section quantifies year-to-year stability in the main behavioral outcomes.

The goal is to assess:

- how strongly each outcome correlates with itself across adjacent years
- how large typical within-dog year-to-year shifts are
- whether the outcomes are mostly fixed, moderately stable, or substantially dynamic across annual assessments

Table 3. Year-to-year stability of the main behavioral outcomes across adjacent annual transitions.

In [15]:

```python
# Step 7. Stability analysis for outcomes

from scipy.stats import wilcoxon, rankdata

outcome_vars = [
    "score_nonsocial_fear",
    "score_separation_related_problems",
    "score_touch_sensitivity",
    "stress_related_comp",
    "fear_composite",
]

def matched_rank_biserial_from_diff(diff):
    """
    Compute matched-pairs rank-biserial correlation from paired differences.
    Zeros are removed before ranking.
    """
    diff = pd.Series(diff).dropna()
    diff = diff[diff != 0]

    if len(diff) == 0:
        return np.nan

    ranks = rankdata(np.abs(diff))
    pos_rank_sum = ranks[diff > 0].sum()
    neg_rank_sum = ranks[diff < 0].sum()

    return (pos_rank_sum - neg_rank_sum) / (pos_rank_sum + neg_rank_sum)

outcome_stability_results = []

for var in outcome_vars:
    for baseline_year in [0, 1]:
        sub = outcome_pairs[
            outcome_pairs["year_in_study"] == baseline_year
        ][[var, f"{var}_next", f"delta_{var}"]].dropna().copy()

        n = len(sub)

        if n >= 20:
            r = sub[var].corr(sub[f"{var}_next"])
        else:
            r = np.nan

        diff = sub[f"delta_{var}"]

        if len(diff[diff != 0]) > 0:
            wilcox = wilcoxon(diff)
            p_value = wilcox.pvalue
            rank_biserial = matched_rank_biserial_from_diff(diff)
        else:
            p_value = np.nan
            rank_biserial = np.nan

        outcome_stability_results.append({
```

```
            "variable": var,
            "transition": f"{baseline_year}->{baseline_year+1}",
            "n_pairs": n,
            "pearson_r": r,
            "mean_delta": diff.mean(),
            "median_delta": diff.median(),
            "mean_abs_delta": diff.abs().mean(),
            "wilcoxon_p": p_value,
            "rank_biserial": rank_biserial,
        })

outcome_stability_df = pd.DataFrame(outcome_stability_results)

outcome_stability_df = outcome_stability_df[
    [
        "variable",
        "transition",
        "n_pairs",
        "pearson_r",
        "mean_delta",
        "median_delta",
        "mean_abs_delta",
        "wilcoxon_p",
        "rank_biserial",
    ]
].sort_values(["variable", "transition"])

display(outcome_stability_df.round(4))
```

	variable	transition	n_pairs	pearson_r	mean_delta	median
8	fear_composite	0->1	2689	0.6417	0.0417	
9	fear_composite	1->2	2486	0.7266	0.0279	
0	score_nonsocial_fear	0->1	2642	0.6339	0.0413	
1	score_nonsocial_fear	1->2	2472	0.7357	0.0314	
2	score_separation_related_problems	0->1	2604	0.5322	-0.0387	
3	score_separation_related_problems	1->2	2415	0.5288	-0.0191	
4	score_touch_sensitivity	0->1	2569	0.5528	0.0407	
5	score_touch_sensitivity	1->2	2393	0.6736	0.0307	
6	stress_related_comp	0->1	2702	0.6384	0.0146	
7	stress_related_comp	1->2	2496	0.7405	0.0154	

The outcome stability results show that fear- and stress-related measures were moderately stable across consecutive years, with year-to-year correlations generally ranging from about 0.53 to 0.74.

At the same time, several outcomes showed small but statistically reliable directional shifts. Non-social fear, touch sensitivity, fear composite, and the stress-related

composite tended to increase slightly over time, whereas separation-related problems tended to decrease. These effects were small in magnitude, indicating systematic but modest year-to-year change rather than large shifts.

Interim interpretation of outcome stability

The main fear- and stress-related outcomes showed moderate year-to-year stability.

Rank-order stability was highest for:

- fear_composite
- score_nonsocial_fear
- stress_related_comp

and somewhat lower, but still substantial, for:

- score_touch_sensitivity
- score_separation_related_problems

At the same time, mean absolute year-to-year changes were clearly above zero for all outcomes, indicating that the outcomes were not static.

Together, these results suggest that the behavioral outcomes are best characterized as moderately stable with gradual annual drift, rather than as either fixed traits or highly volatile states.

In [16]:

```
# Compact stability plot for outcomes
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Correlations
for var in outcome_vars:
    sub = outcome_stability_df[outcome_stability_df["variable"] == var]
    axes[0].plot(sub["transition"], sub["pearson_r"], marker="o", label=var)

axes[0].set_title("Outcome stability correlations")
axes[0].set_ylabel("Pearson r")
axes[0].set_ylim(0, 1)
axes[0].legend(bbox_to_anchor=(1.05, 1), loc="upper left")

# Mean absolute change
for var in outcome_vars:
    sub = outcome_stability_df[outcome_stability_df["variable"] == var]
    axes[1].plot(sub["transition"], sub["mean_abs_delta"], marker="o", label

axes[1].set_title("Outcome mean absolute change")
axes[1].set_ylabel("Mean absolute delta")

plt.tight_layout()
plt.show()
```

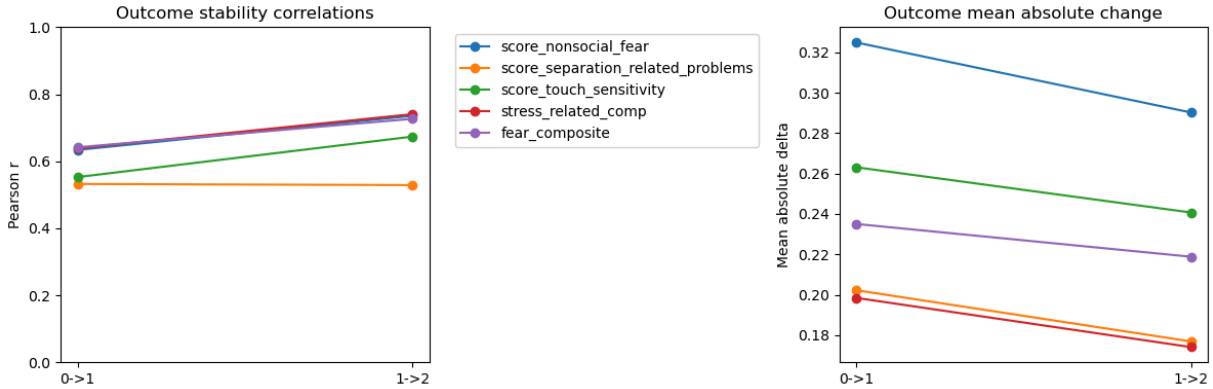


Figure 5. Compact summary of year-to-year stability for the main behavioral outcomes, showing Pearson correlations and mean absolute change across adjacent annual transitions.

Table 4. Direction of year-to-year change in the main behavioral outcomes across adjacent annual transitions.

In [17]:
```
# Direction of year-to-year change in outcomes

outcome_direction_results = []

for var in outcome_vars:
    for baseline_year in [0, 1]:
        sub = outcome_pairs[
            outcome_pairs["year_in_study"] == baseline_year
        ][f"delta_{var}"].dropna()

        n = len(sub)
        n_increased = (sub > 0).sum()
        n_decreased = (sub < 0).sum()
        n_unchanged = (sub == 0).sum()

        outcome_direction_results.append({
            "variable": var,
            "transition": f"{baseline_year}->{baseline_year+1}",
            "n_pairs": n,
            "n_increased": n_increased,
            "pct_increased": n_increased / n * 100 if n > 0 else np.nan,
            "n_decreased": n_decreased,
            "pct_decreased": n_decreased / n * 100 if n > 0 else np.nan,
            "n_unchanged": n_unchanged,
            "pct_unchanged": n_unchanged / n * 100 if n > 0 else np.nan,
        })

outcome_direction_df = pd.DataFrame(outcome_direction_results)

display(outcome_direction_df.round(2))
```

	variable	transition	n_pairs	n_increased	pct_increased	n_
0	score_nonsocial_fear	0->1	2642	1102	41.71	
1	score_nonsocial_fear	1->2	2472	1011	40.90	
2	score_separation_related_problems	0->1	2604	646	24.81	
3	score_separation_related_problems	1->2	2415	567	23.48	
4	score_touch_sensitivity	0->1	2569	809	31.49	
5	score_touch_sensitivity	1->2	2393	712	29.75	
6	stress_related_comp	0->1	2702	1294	47.89	
7	stress_related_comp	1->2	2496	1195	47.88	
8	fear_composite	0->1	2689	1301	48.38	
9	fear_composite	1->2	2486	1186	47.71	

The direction-of-change table shows that statistically reliable year-to-year shifts do not imply uniform change across all dogs.

For several outcomes, increases were more common than decreases, but the differences were modest rather than overwhelming. This indicates that the overall upward shifts in fear-related measures reflect a small aggregate tendency across the sample, not a universal increase in every dog. Separation-related problems showed the opposite pattern, with decreases occurring more often than increases.

Table 5. Magnitude of year-to-year change in the main behavioral outcomes among dogs with non-zero change.

In [18]:
```
# Magnitude of outcome change among dogs with non-zero change

outcome_change_magnitude_results = []

for var in outcome_vars:
    for baseline_year in [0, 1]:
        sub = outcome_pairs[
            outcome_pairs["year_in_study"] == baseline_year
        ][f"delta_{var}"].dropna()

        increased = sub[sub > 0]
        decreased = sub[sub < 0]

        outcome_change_magnitude_results.append({
            "variable": var,
            "transition": f"{baseline_year}->{baseline_year+1}",
            "n_pairs": len(sub),
            "n_changed": (sub != 0).sum(),
            "pct_changed": ((sub != 0).sum() / len(sub) * 100) if len(sub) >
            "n_increased": len(increased),
            "mean_increase": increased.mean() if len(increased) > 0 else np.
```

```
            "median_increase": increased.median() if len(increased) > 0 else
            "n_decreased": len(decreased),
            "mean_decrease": decreased.mean() if len(decreased) > 0 else np.
            "median_decrease": decreased.median() if len(decreased) > 0 else
            "mean_abs_change_among_changers": sub[sub != 0].abs().mean() if
            "median_abs_change_among_changers": sub[sub != 0].abs().median()
        })

outcome_change_magnitude_df = pd.DataFrame(outcome_change_magnitude_results)

display(outcome_change_magnitude_df.round(4))
```

	variable	transition	n_pairs	n_changed	pct_changed	n_in
0	score_nonsocial_fear	0->1	2642	2077	78.6147	
1	score_nonsocial_fear	1->2	2472	1865	75.4450	
2	score_separation_related_problems	0->1	2604	1592	61.1367	
3	score_separation_related_problems	1->2	2415	1291	53.4576	
4	score_touch_sensitivity	0->1	2569	1458	56.7536	
5	score_touch_sensitivity	1->2	2393	1296	54.1580	
6	stress_related_comp	0->1	2702	2499	92.4870	
7	stress_related_comp	1->2	2496	2253	90.2644	
8	fear_composite	0->1	2689	2408	89.5500	
9	fear_composite	1->2	2486	2213	89.0185	

The magnitude table shows that, among dogs whose scores changed, year-to-year shifts were usually modest rather than large.

For individual scales, typical changes were often around 0.25 to 0.33 points in either direction. Increases and decreases were generally similar in size, suggesting that the overall directional trends were driven more by how often scores increased versus decreased than by dramatically larger changes in one direction. Composite outcomes changed more frequently, as expected, because small shifts in any component scale alter the overall mean.

In [19]:
```
# Stacked bar plot of outcome change direction

plot_df = outcome_direction_df.copy()

plot_df = plot_df[
    ["variable", "transition", "pct_increased", "pct_decreased", "pct_unchar
].copy()

plot_df["group"] = plot_df["variable"] + " (" + plot_df["transition"] + ")"
plot_df = plot_df.set_index("group")
```

```
ax = plot_df[["pct_decreased", "pct_unchanged", "pct_increased"]].plot(
    kind="bar",
    stacked=True,
    figsize=(10, 6)
)

ax.set_title("Direction of year-to-year change in outcomes")
ax.set_xlabel("Outcome and transition")
ax.set_ylabel("Percent of dog pairs")
plt.xticks(rotation=45, ha="right")
plt.legend(title="Change direction", bbox_to_anchor=(1.05, 1), loc="upper le
plt.tight_layout()
plt.show()
```

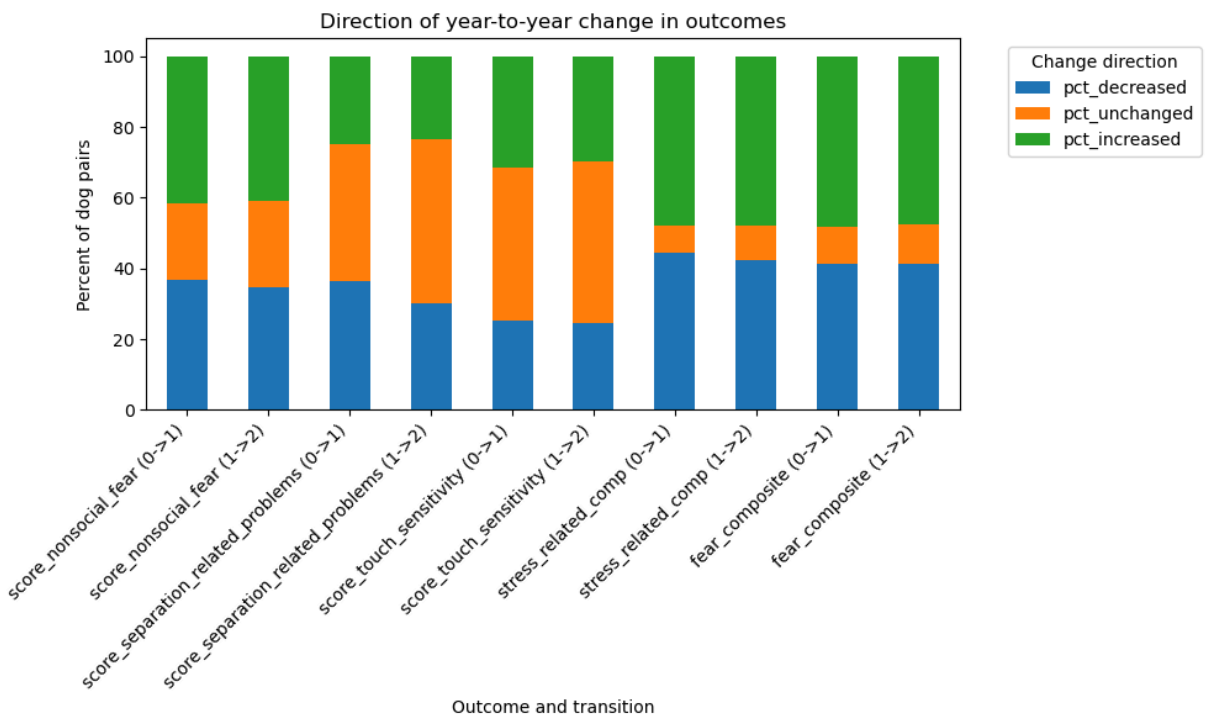


Figure 6. Direction of year-to-year change in the main behavioral outcomes across adjacent annual transitions.

Step 8. Stability analysis for predictors

This section quantifies year-to-year stability in the main walk and activity predictors.

The goal is to assess:

- how strongly each predictor correlates with itself across adjacent years
- how large typical within-dog year-to-year shifts are
- whether the predictors behave as relatively stable routine variables or as highly changeable exposures

Table 6. Year-to-year stability of the main walking and activity predictors across adjacent annual transitions.

In [20]:
```
# Step 8. Stability analysis for predictors

from scipy.stats import wilcoxon, rankdata

predictor_vars = [
    "activity_level_num",
    "walk_frequency_num",
    "walk_without_leash_num",
    "offleash_dose_index_filled",
    "fetch_games_frequency_num",
]

def matched_rank_biserial_from_diff(diff):
    """
    Compute matched-pairs rank-biserial correlation from paired differences.
    Zeros are removed before ranking.
    """
    diff = pd.Series(diff).dropna()
    diff = diff[diff != 0]

    if len(diff) == 0:
        return np.nan

    ranks = rankdata(np.abs(diff))
    pos_rank_sum = ranks[diff > 0].sum()
    neg_rank_sum = ranks[diff < 0].sum()

    return (pos_rank_sum - neg_rank_sum) / (pos_rank_sum + neg_rank_sum)

predictor_stability_results = []

for var in predictor_vars:
    for baseline_year in [0, 1]:
        sub = predictor_pairs[
            predictor_pairs["year_in_study"] == baseline_year
        ][[var, f"{var}_next", f"delta_{var}"]].dropna().copy()

        n = len(sub)

        if n >= 20:
            r = sub[var].corr(sub[f"{var}_next"])
        else:
            r = np.nan

        diff = sub[f"delta_{var}"]

        if len(diff[diff != 0]) > 0:
            wilcox = wilcoxon(diff)
            p_value = wilcox.pvalue
            rank_biserial = matched_rank_biserial_from_diff(diff)
        else:
            p_value = np.nan
```

```
            rank_biserial = np.nan

        predictor_stability_results.append({
            "variable": var,
            "transition": f"{baseline_year}->{baseline_year+1}",
            "n_pairs": n,
            "pearson_r": r,
            "mean_delta": diff.mean(),
            "median_delta": diff.median(),
            "mean_abs_delta": diff.abs().mean(),
            "wilcoxon_p": p_value,
            "rank_biserial": rank_biserial,
        })

predictor_stability_df = pd.DataFrame(predictor_stability_results)

predictor_stability_df = predictor_stability_df[
    [
        "variable",
        "transition",
        "n_pairs",
        "pearson_r",
        "mean_delta",
        "median_delta",
        "mean_abs_delta",
        "wilcoxon_p",
        "rank_biserial",
    ]
].sort_values(["variable", "transition"])

display(predictor_stability_df.round(4))
```

	variable	transition	n_pairs	pearson_r	mean_delta	median_delta
0	activity_level_num	0->1	2704	0.5091	-0.0817	0.0
1	activity_level_num	1->2	2496	0.5909	-0.1114	0.0
8	fetch_games_frequency_num	0->1	2705	0.4613	-0.0100	0.0
9	fetch_games_frequency_num	1->2	2497	0.6191	-0.0729	0.0
6	offleash_dose_index_filled	0->1	2705	0.6452	0.0249	0.0
7	offleash_dose_index_filled	1->2	2497	0.6850	-0.0259	0.0
2	walk_frequency_num	0->1	2705	0.6458	-0.4133	0.0
3	walk_frequency_num	1->2	2497	0.7167	-0.1322	0.0
4	walk_without_leash_num	0->1	2705	0.6178	0.0429	0.0
5	walk_without_leash_num	1->2	2497	0.6604	-0.0088	0.0

The predictor stability results indicate that annual walk/activity measures were also moderately stable across consecutive years, with year-to-year correlations generally in the moderate range.

At the same time, several predictors showed small but meaningful directional shifts. Activity level and walk frequency tended to decline over time, while off-leash access showed a slight increase from Year 0 to Year 1 and then remained relatively stable. Overall, the results support treating these annual measures as routine profiles that are fairly stable but still capable of changing within dogs over time.

Table 7. Direction of year-to-year change in the main walking and activity predictors across adjacent annual transitions.

In [21]:
```
# Direction of year-to-year change in predictors

predictor_direction_results = []

for var in predictor_vars:
    for baseline_year in [0, 1]:
        sub = predictor_pairs[
            predictor_pairs["year_in_study"] == baseline_year
        ][f"delta_{var}"].dropna()

        n = len(sub)
        n_increased = (sub > 0).sum()
        n_decreased = (sub < 0).sum()
        n_unchanged = (sub == 0).sum()

        predictor_direction_results.append({
            "variable": var,
            "transition": f"{baseline_year}->{baseline_year+1}",
            "n_pairs": n,
            "n_increased": n_increased,
            "pct_increased": n_increased / n * 100 if n > 0 else np.nan,
            "n_decreased": n_decreased,
            "pct_decreased": n_decreased / n * 100 if n > 0 else np.nan,
            "n_unchanged": n_unchanged,
            "pct_unchanged": n_unchanged / n * 100 if n > 0 else np.nan,
        })

predictor_direction_df = pd.DataFrame(predictor_direction_results)

display(predictor_direction_df.round(2))
```

	variable	transition	n_pairs	n_increased	pct_increased	n_decre
0	activity_level_num	0->1	2704	263	9.73	
1	activity_level_num	1->2	2496	168	6.73	
2	walk_frequency_num	0->1	2705	352	13.01	
3	walk_frequency_num	1->2	2497	423	16.94	
4	walk_without_leash_num	0->1	2705	318	11.76	
5	walk_without_leash_num	1->2	2497	201	8.05	
6	offleash_dose_index_filled	0->1	2705	671	24.81	
7	offleash_dose_index_filled	1->2	2497	549	21.99	
8	fetch_games_frequency_num	0->1	2705	377	13.94	
9	fetch_games_frequency_num	1->2	2497	207	8.29	

Table 8. Magnitude of year-to-year change in the main walking and activity predictors among dogs with non-zero change.

In [22]:
```
# Magnitude of predictor change among dogs with non-zero change

predictor_change_magnitude_results = []

for var in predictor_vars:
    for baseline_year in [0, 1]:
        sub = predictor_pairs[
            predictor_pairs["year_in_study"] == baseline_year
        ][f"delta_{var}"].dropna()

        increased = sub[sub > 0]
        decreased = sub[sub < 0]

        predictor_change_magnitude_results.append({
            "variable": var,
            "transition": f"{baseline_year}->{baseline_year+1}",
            "n_pairs": len(sub),
            "n_changed": (sub != 0).sum(),
            "pct_changed": ((sub != 0).sum() / len(sub) * 100) if len(sub) >
            "n_increased": len(increased),
            "mean_increase": increased.mean() if len(increased) > 0 else np.
            "median_increase": increased.median() if len(increased) > 0 else
            "n_decreased": len(decreased),
            "mean_decrease": decreased.mean() if len(decreased) > 0 else np.
            "median_decrease": decreased.median() if len(decreased) > 0 else
            "mean_abs_change_among_changers": sub[sub != 0].abs().mean() if
            "median_abs_change_among_changers": sub[sub != 0].abs().median()
        })

predictor_change_magnitude_df = pd.DataFrame(predictor_change_magnitude_resu

display(predictor_change_magnitude_df.round(4))
```

	variable	transition	n_pairs	n_changed	pct_changed	n_increase
0	activity_level_num	0->1	2704	741	27.4038	26
1	activity_level_num	1->2	2496	613	24.5593	16
2	walk_frequency_num	0->1	2705	1199	44.3253	35
3	walk_frequency_num	1->2	2497	1011	40.4886	42
4	walk_without_leash_num	0->1	2705	520	19.2237	31
5	walk_without_leash_num	1->2	2497	424	16.9804	20
6	offleash_dose_index_filled	0->1	2705	1334	49.3161	67
7	offleash_dose_index_filled	1->2	2497	1214	48.6183	54
8	fetch_games_frequency_num	0->1	2705	783	28.9464	37
9	fetch_games_frequency_num	1->2	2497	596	23.8686	20

The magnitude table shows that, among dogs whose routine variables changed, those changes were often discrete and meaningful rather than minimal.

For most ordinal predictors, the typical shift was about one category in either direction. Walk frequency stood out in particular: among dogs who changed, the median shift was about two categories, indicating that changes in walk frequency were often substantial when they occurred. Off-leash dose showed a different pattern, with more gradual shifts in magnitude, consistent with its construction as a continuous-like index rather than a simple category.

In [23]:
```
# Stacked bar plot of predictor change direction

plot_df = predictor_direction_df.copy()

plot_df = plot_df[
    ["variable", "transition", "pct_increased", "pct_decreased", "pct_unchar
].copy()

plot_df["group"] = plot_df["variable"] + " (" + plot_df["transition"] + ")"
plot_df = plot_df.set_index("group")

ax = plot_df[["pct_decreased", "pct_unchanged", "pct_increased"]].plot(
    kind="bar",
    stacked=True,
    figsize=(10, 6)
)

ax.set_title("Direction of year-to-year change in predictors")
ax.set_xlabel("Predictor and transition")
ax.set_ylabel("Percent of dog pairs")
plt.xticks(rotation=45, ha="right")
plt.legend(title="Change direction", bbox_to_anchor=(1.05, 1), loc="upper le
plt.tight_layout()
plt.show()
```

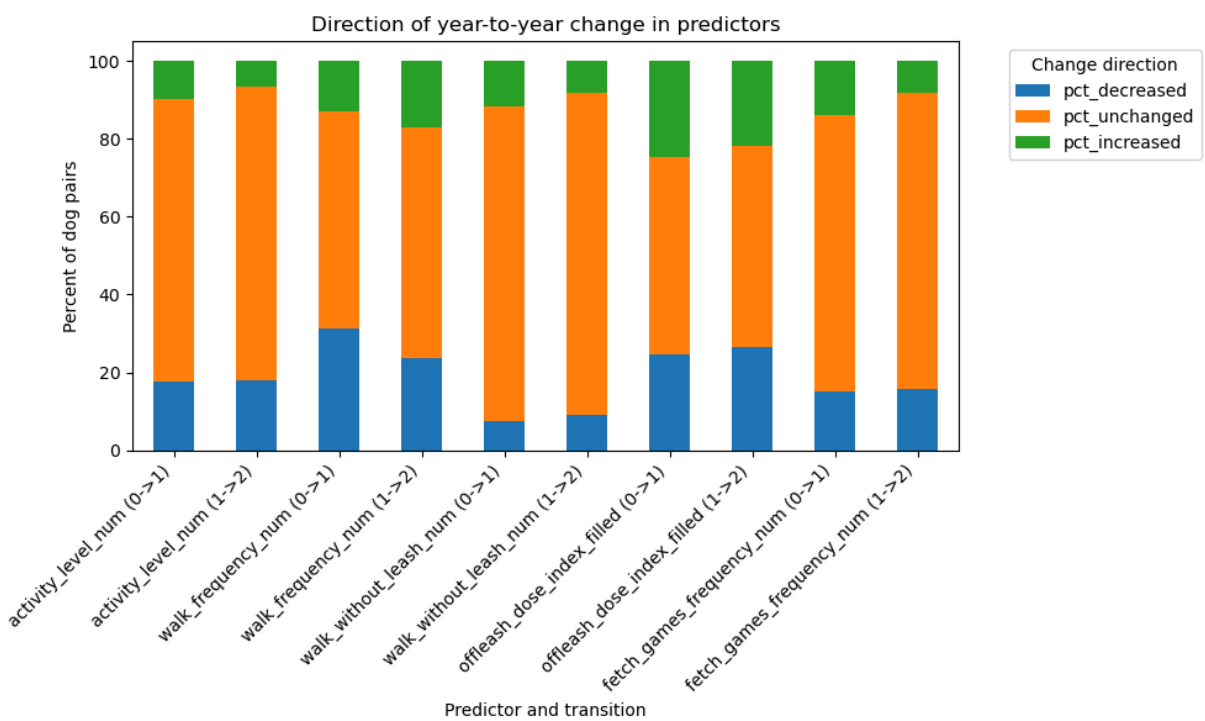


Figure 7. Direction of year-to-year change in the main walking and activity predictors across adjacent annual transitions.

In [24]:
```
# Compact stability plot for predictors

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Correlations
for var in predictor_vars:
    sub = predictor_stability_df[predictor_stability_df["variable"] == var]
    axes[0].plot(sub["transition"], sub["pearson_r"], marker="o", label=var)

axes[0].set_title("Predictor stability correlations")
axes[0].set_ylabel("Pearson r")
axes[0].set_ylim(0, 1)
axes[0].legend(bbox_to_anchor=(1.05, 1), loc="upper left")

# Mean absolute change
for var in predictor_vars:
    sub = predictor_stability_df[predictor_stability_df["variable"] == var]
    axes[1].plot(sub["transition"], sub["mean_abs_delta"], marker="o", label

axes[1].set_title("Predictor mean absolute change")
axes[1].set_ylabel("Mean absolute delta")

plt.tight_layout()
plt.show()
```

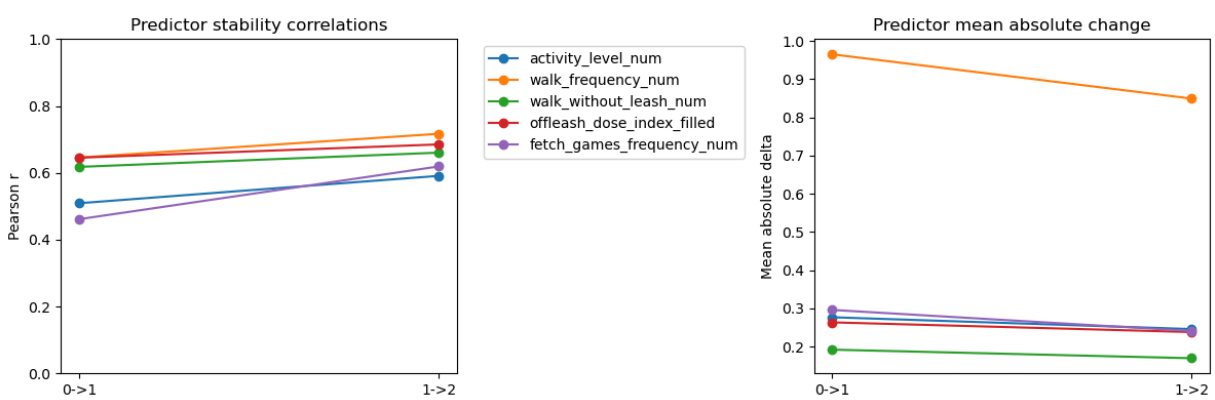


Figure 8. Compact summary of year-to-year stability for the main walking and activity predictors, showing Pearson correlations and mean absolute change across adjacent annual transitions.

Interim interpretation of predictor stability

The main walk and activity predictors showed moderate year-to-year stability.

Stability was strongest for:

- walk_frequency_num
- walk_without_leash_num

- offleash_dose_index_filled

and somewhat lower for:

- activity_level_num
- fetch_games_frequency_num

At the same time, mean absolute changes were clearly above zero, indicating that the predictors were not static.

Together, these results suggest that the walk/activity variables are best interpreted as relatively stable routine measures with modest annual drift.

The predictor stability results indicate that the main walk/activity variables are moderately stable across adjacent annual assessments.

Stability was strongest for walk frequency, off-leash access, and off-leash dose, suggesting that these variables capture relatively persistent routine patterns rather than purely transient exposures.

At the same time, non-zero mean absolute change indicates that annual drift is present, especially for walk frequency and activity level.

Step 9. Build calibration and validation split by dog

This section creates the calibration and validation framework for the main modeling steps.

The goal is to:

- split the dataset by dog rather than by row
- prevent leakage across repeated observations from the same dog
- create a calibration subset for model fitting
- reserve an independent validation subset for later model assessment

In [25]:
```
# Step 9. Build calibration and validation split by dog

from sklearn.model_selection import train_test_split

# Use unique dogs to avoid leakage across repeated observations
all_dogs = yearly_analysis_df["subject_id"].dropna().unique()

train_dogs, test_dogs = train_test_split(
    all_dogs,
    test_size=0.30,
    random_state=42,
    shuffle=True
)
```

```python
train_dogs = set(train_dogs)
test_dogs = set(test_dogs)

# Apply split to yearly dataset
yearly_train = yearly_analysis_df[
    yearly_analysis_df["subject_id"].isin(train_dogs)
].copy()

yearly_test = yearly_analysis_df[
    yearly_analysis_df["subject_id"].isin(test_dogs)
].copy()

print("Total dogs:", len(all_dogs))
print("Train dogs:", len(train_dogs))
print("Test dogs:", len(test_dogs))

print("\nYearly train shape:", yearly_train.shape)
print("Yearly test shape:", yearly_test.shape)

# Sanity check: no dog should appear in both sets
overlap_dogs = set(yearly_train["subject_id"]).intersection(set(yearly_test[
print("\nDog overlap between train and test:", len(overlap_dogs))

print("\nRows by year in train:")
display(yearly_train["year_in_study"].value_counts().sort_index())

print("\nRows by year in test:")
display(yearly_test["year_in_study"].value_counts().sort_index())

print("\nUnique dogs by year in train:")
display(yearly_train.groupby("year_in_study")["subject_id"].nunique())

print("\nUnique dogs by year in test:")
display(yearly_test.groupby("year_in_study")["subject_id"].nunique())
```

```
Total dogs: 3044
Train dogs: 2130
Test dogs: 914

Yearly train shape: (5865, 26)
Yearly test shape: (2487, 26)

Dog overlap between train and test: 0

Rows by year in train:
year_in_study
0    2130
1    1902
2    1833
Name: count, dtype: int64
Rows by year in test:
year_in_study
0    914
1    803
2    770
Name: count, dtype: int64
```

```
Unique dogs by year in train:
year_in_study
0    2130
1    1902
2    1833
Name: subject_id, dtype: int64
Unique dogs by year in test:
year_in_study
0    914
1    803
2    770
Name: subject_id, dtype: int64
```

The dataset was split at the dog level into calibration (training) and validation (test) subsets.

This split ensures that all repeated observations from the same dog remain in the same subset, preventing leakage across years. Both subsets retain observations from all three study years, allowing later models to be fit on the calibration set and evaluated on an independent validation set of dogs.

Step 10. Same-year mixed models

This section tests whether annual walk/activity routine variables are associated with fear- and stress-related outcomes within the same study year.

The goal is to:

- estimate same-year associations between routine variables and behavioral outcomes
- control for dog age and study year
- account for repeated observations within dogs
- identify which walk/activity variables show consistent cross-sectional associations before moving to lagged models

In [26]:

```
# Step 10. Same-year mixed models

import warnings
import pandas as pd
import statsmodels.formula.api as smf
from IPython.display import display, Markdown

# ---------------------------------------------------
# 1. Define outcomes and predictors
# ---------------------------------------------------
same_year_outcomes = [
    "score_nonsocial_fear",
    "score_separation_related_problems",
    "score_touch_sensitivity",
    "stress_related_comp",
```

```
    "fear_composite",
]

same_year_predictors = [
    "activity_level_num",
    "walk_frequency_num",
    "walk_without_leash_num",
    "offleash_dose_index_filled",
    "fetch_games_frequency_num",
]

outcome_labels = {
    "score_nonsocial_fear": "Nonsocial fear",
    "score_separation_related_problems": "Separation-related problems",
    "score_touch_sensitivity": "Touch sensitivity",
    "stress_related_comp": "Stress-related composite",
    "fear_composite": "Fear composite",
}

predictor_labels = {
    "age_years": "Age (years)",
    "activity_level_num": "Overall activity level",
    "walk_frequency_num": "Walk frequency",
    "walk_without_leash_num": "Walking without leash",
    "offleash_dose_index_filled": "Off-leash activity index",
    "fetch_games_frequency_num": "Fetch-game frequency",
}

# ---------------------------------------------------
# 2. Fit main same-year models
# ---------------------------------------------------
same_year_results = []
same_year_models = []

for outcome in same_year_outcomes:
    model_vars = [
        "subject_id",
        "year_in_study",
        "age_years",
        outcome,
    ] + same_year_predictors

    df_model = yearly_train[model_vars].dropna().copy()

    formula = (
        f"{outcome} ~ age_years + C(year_in_study) + "
        + " + ".join(same_year_predictors)
    )

    try:
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            model = smf.mixedlm(
                formula=formula,
                data=df_model,
                groups=df_model["subject_id"]
```

```
            )
            fit = model.fit(reml=False, method="lbfgs", maxiter=200)

        params = fit.params
        pvalues = fit.pvalues
        conf_int = fit.conf_int()

        for pred in ["age_years"] + same_year_predictors:
            if pred in params.index:
                same_year_results.append({
                    "Outcome": outcome_labels[outcome],
                    "Predictor": predictor_labels[pred],
                    "Coefficient": params[pred],
                    "p-value": pvalues[pred],
                    "CI lower": conf_int.loc[pred, 0],
                    "CI upper": conf_int.loc[pred, 1],
                    "Rows used": df_model.shape[0],
                })

    except Exception:
        # Separation-related problems will be handled in a follow-up block b
        continue

same_year_results_df = pd.DataFrame(same_year_results).round(4)

# ----------------------------------------------------
# 3. Display compact coefficient table
# ----------------------------------------------------
display(Markdown(
    "**Table 9.** Same-year mixed-effects model coefficients for associatior
))
display(same_year_results_df)

# ----------------------------------------------------
# 4. Follow-up model for separation-related problems
# ----------------------------------------------------
sep_outcome = "score_separation_related_problems"

sep_predictors = [
    "activity_level_num",
    "walk_frequency_num",
    "walk_without_leash_num",
    "offleash_dose_index_filled",
    "fetch_games_frequency_num",
]

sep_vars = [
    "subject_id",
    "year_in_study",
    "age_years",
    sep_outcome,
] + sep_predictors

sep_df = yearly_train[sep_vars].dropna().copy()

sep_formula = (
```

```
    f"{sep_outcome} ~ age_years + C(year_in_study) + "
    + " + ".join(sep_predictors)
)

sep_fit = None
sep_optimizer = None

for method in ["powell", "cg", "lbfgs"]:
    try:
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            sep_model = smf.mixedlm(
                formula=sep_formula,
                data=sep_df,
                groups=sep_df["subject_id"]
            )
            sep_fit = sep_model.fit(reml=False, method=method, maxiter=300)
        sep_optimizer = method
        break
    except Exception:
        continue

if sep_fit is not None:
    sep_results_df = pd.DataFrame({
        "Predictor": sep_fit.params.index,
        "Coefficient": sep_fit.params.values,
        "p-value": sep_fit.pvalues.values,
        "CI lower": sep_fit.conf_int()[0].values,
        "CI upper": sep_fit.conf_int()[1].values,
    }).round(4)

    keep_rows = [
        "C(year_in_study)[T.1]",
        "C(year_in_study)[T.2]",
        "age_years",
        "activity_level_num",
        "walk_frequency_num",
        "walk_without_leash_num",
        "offleash_dose_index_filled",
        "fetch_games_frequency_num",
    ]
    sep_results_df = sep_results_df[sep_results_df["Predictor"].isin(keep_rc

    sep_predictor_display = {
        "C(year_in_study)[T.1]": "Study year = 1",
        "C(year_in_study)[T.2]": "Study year = 2",
        "age_years": "Age (years)",
        "activity_level_num": "Overall activity level",
        "walk_frequency_num": "Walk frequency",
        "walk_without_leash_num": "Walking without leash",
        "offleash_dose_index_filled": "Off-leash activity index",
        "fetch_games_frequency_num": "Fetch-game frequency",
    }
    sep_results_df["Predictor"] = sep_results_df["Predictor"].map(sep_predic
    sep_results_df = sep_results_df.reset_index(drop=True)
```

```
    display(Markdown(
        f"The same-year mixed-effects model for separation-related problems
    ))
    display(Markdown(
        "**Table 10.** Follow-up same-year mixed-effects model coefficients
    ))
    display(sep_results_df)
else:
    display(Markdown(
        "The follow-up same-year mixed-effects model for separation-related
    ))


# ---------------------------------------------------
# 5. Save quietly
# ---------------------------------------------------
same_year_results_df.to_csv("step10_same_year_mixed_models.csv", index=False
if sep_fit is not None:
    sep_results_df.to_csv("step10_separation_followup_model.csv", index=Fals
```

Table 9. Same-year mixed-effects model coefficients for associations between annual walking/activity predictors and concurrent behavioral outcomes.

	Outcome	Predictor	Coefficient	p-value	CI lower	CI upper	Rows used
0	Nonsocial fear	Age (years)	0.0307	0.1083	-0.0068	0.0681	5799
1	Nonsocial fear	Overall activity level	-0.0362	0.0055	-0.0617	-0.0106	5799
2	Nonsocial fear	Walk frequency	0.0035	0.3363	-0.0037	0.0108	5799
3	Nonsocial fear	Walking without leash	0.0200	0.4825	-0.0359	0.0759	5799
4	Nonsocial fear	Off-leash activity index	-0.0319	0.2382	-0.0849	0.0211	5799
5	Nonsocial fear	Fetch-game frequency	-0.0203	0.0973	-0.0444	0.0037	5799
6	Touch sensitivity	Age (years)	0.0041	0.7897	-0.0262	0.0345	5707
7	Touch sensitivity	Overall activity level	-0.0114	0.3136	-0.0337	0.0108	5707
8	Touch sensitivity	Walk frequency	0.0020	0.5355	-0.0042	0.0082	5707
9	Touch sensitivity	Walking without leash	-0.0029	0.9081	-0.0523	0.0465	5707
10	Touch sensitivity	Off-leash activity index	-0.0222	0.3530	-0.0691	0.0247	5707
11	Touch sensitivity	Fetch-game frequency	-0.0240	0.0259	-0.0451	-0.0029	5707
12	Stress-related composite	Age (years)	0.0087	0.4616	-0.0144	0.0317	5858
13	Stress-related composite	Overall activity level	-0.0171	0.0316	-0.0326	-0.0015	5858
14	Stress-related composite	Walk frequency	0.0020	0.3746	-0.0024	0.0064	5858
15	Stress-related composite	Walking without leash	0.0021	0.9031	-0.0320	0.0362	5858
16	Stress-related composite	Off-leash activity index	-0.0183	0.2688	-0.0507	0.0141	5858
17	Stress-related composite	Fetch-game frequency	-0.0165	0.0276	-0.0312	-0.0018	5858
18	Fear composite	Age (years)	0.0245	0.0847	-0.0034	0.0524	5842
19	Fear composite	Overall activity level	-0.0344	0.0003	-0.0533	-0.0156	5842
20	Fear composite	Walk frequency	0.0018	0.4989	-0.0035	0.0072	5842
21	Fear composite	Walking without leash	-0.0149	0.4805	-0.0562	0.0264	5842

	Outcome	Predictor	Coefficient	p-value	CI lower	CI upper	Rows used
22	Fear composite	Off-leash activity index	-0.0045	0.8213	-0.0437	0.0347	5842
23	Fear composite	Fetch-game frequency	-0.0047	0.6003	-0.0225	0.0130	5842

The same-year mixed-effects model for separation-related problems converged in a follow-up fit using the **powell** optimizer.

Table 10. Follow-up same-year mixed-effects model coefficients for separation-related problems.

	Predictor	Coefficient	p-value	CI lower	CI upper
0	Study year = 1	-0.0216	0.1061	-0.0479	0.0046
1	Study year = 2	-0.0269	0.2494	-0.0726	0.0188
2	Age (years)	-0.0145	0.1898	-0.0362	0.0072
3	Overall activity level	-0.0057	0.5160	-0.0229	0.0115
4	Walk frequency	0.0012	0.6127	-0.0035	0.0059
5	Walking without leash	0.0159	0.4128	-0.0221	0.0539
6	Off-leash activity index	-0.0190	0.3022	-0.0550	0.0171
7	Fetch-game frequency	-0.0132	0.1109	-0.0295	0.0030

The same-year mixed models suggested that cross-sectional associations between annual routine variables and fear- and stress-related outcomes were generally modest. Across the converged models, higher overall activity level showed the most consistent association with lower fear-related scores, whereas more frequent fetch games were associated with slightly lower touch sensitivity and lower stress-related composite scores. By contrast, walk frequency, walking without leash, and the off-leash activity index did not show strong independent same-year associations in the adjusted models. Age and study-year effects were generally weak after accounting for the routine predictors and repeated observations within dogs. The same-year model for separation-related problems required an alternative optimizer to converge and, after convergence, did not show clear cross-sectional associations with the routine predictors.

Step 11. Build the lagged analysis dataset

This section constructs the lagged dataset used for the main longitudinal models.

The goal is to:

- pair each dog's outcome at time `t` with the same outcome at time `t+1`

- retain the routine predictors measured at time `t`
- carry forward age and study-wave information
- create a clean dataset for testing whether current routine variables are associated with subsequent behavioral outcomes

In [27]:
```
# Step 11. Build the lagged analysis dataset

lagged_outcomes = [
    "score_nonsocial_fear",
    "score_separation_related_problems",
    "score_touch_sensitivity",
    "stress_related_comp",
    "fear_composite",
]

lagged_predictors = [
    "activity_level_num",
    "walk_frequency_num",
    "walk_without_leash_num",
    "offleash_dose_index_filled",
    "fetch_games_frequency_num",
]

lagged_vars = (
    ["subject_id", "year_in_study", "age_years"]
    + lagged_outcomes
    + lagged_predictors
)

lagged_base = yearly_analysis_df[lagged_vars].copy()

lagged_next = lagged_base[
    ["subject_id", "year_in_study"] + lagged_outcomes
].copy()

lagged_next["year_in_study"] = lagged_next["year_in_study"] - 1
lagged_next = lagged_next.rename(
    columns={v: f"{v}_next" for v in lagged_outcomes}
)

lagged_analysis_df = lagged_base.merge(
    lagged_next,
    on=["subject_id", "year_in_study"],
    how="inner"
)

# Keep only consecutive pairs 0->1 and 1->2
lagged_analysis_df = lagged_analysis_df[
    lagged_analysis_df["year_in_study"].isin([0, 1])
].copy()

# Rename current-year variables for clarity
lagged_analysis_df = lagged_analysis_df.rename(
    columns={
```

```
        "age_years": "age_years_t",
        "year_in_study": "year_in_study_t",
        **{v: f"{v}_t" for v in lagged_outcomes},
        **{v: f"{v}_t" for v in lagged_predictors},
    }
)

lagged_analysis_df["year_in_study_next"] = lagged_analysis_df["year_in_study

print("Lagged analysis dataset shape:", lagged_analysis_df.shape)
print("Unique dogs in lagged dataset:", lagged_analysis_df["subject_id"].nur
print("Duplicate dog-year rows:", lagged_analysis_df.duplicated(subset=["sub

print("\nRows by transition:")
display(lagged_analysis_df["year_in_study_t"].value_counts().sort_index())

print("\nMissingness in key lagged outcomes:")
display(
    lagged_analysis_df[
        [f"{v}_t" for v in lagged_outcomes] + [f"{v}_next" for v in lagged_c
    ].isna().mean().round(4)
)

print("\nMissingness in key lagged predictors:")
display(
    lagged_analysis_df[
        [f"{v}_t" for v in lagged_predictors] + ["age_years_t", "year_in_stu
    ].isna().mean().round(4)
)

display(lagged_analysis_df.head())
```

```
Lagged analysis dataset shape: (5202, 19)
Unique dogs in lagged dataset: 2705
Duplicate dog-year rows: 0

Rows by transition:
year_in_study_t
0    2705
1    2497
Name: count, dtype: int64
Missingness in key lagged outcomes:
score_nonsocial_fear_t                    0.0123
score_separation_related_problems_t       0.0200
score_touch_sensitivity_t                 0.0281
stress_related_comp_t                     0.0004
fear_composite_t                          0.0029
score_nonsocial_fear_next                 0.0054
score_separation_related_problems_next    0.0185
score_touch_sensitivity_next              0.0252
stress_related_comp_next                  0.0006
fear_composite_next                       0.0025
dtype: float64
Missingness in key lagged predictors:
```

```
activity_level_num_t            0.0002
walk_frequency_num_t            0.0000
walk_without_leash_num_t        0.0000
offleash_dose_index_filled_t    0.0000
fetch_games_frequency_num_t     0.0000
age_years_t                     0.0000
year_in_study_next              0.0000
dtype: float64
```

	subject_id	year_in_study_t	age_years_t	score_nonsocial_fear_t	score_separati
0	grls007MPNZZ	0	1.251198	1.83	
1	grls007MPNZZ	1	2.251198	2.00	
2	grls00NB23WW	0	2.001369	NaN	
3	grls00NB23WW	1	3.001369	0.17	
4	grls01C2L8RR	0	0.495551	0.83	

The outputs above summarize the structure of the lagged analysis dataset, including transition counts and missingness patterns.

The lagged dataset contained consecutive dog-level pairs linking each outcome at time t to the same outcome at time t+1. It retained routine predictors measured at time t, together with age and study-wave information, and served as the analytic dataset for the main longitudinal models. Missingness was negligible for the core predictors and limited for most outcomes.

Step 12. Main lagged mixed models

This section fits the main longitudinal models linking routine variables measured at time `t` to behavioral outcomes at time `t+1`.

The goal is to:

- estimate whether current walk/activity profiles are associated with subsequent fear- and stress-related outcomes
- control for baseline outcome level at time `t`
- account for dog age and study-wave
- model repeated observations within dogs using random intercepts

In [31]:
```
# Step 12. Main lagged mixed models

import warnings
import pandas as pd
import statsmodels.formula.api as smf
from IPython.display import display, Markdown

lagged_outcomes = [
```

```
    "score_nonsocial_fear",
    "score_separation_related_problems",
    "score_touch_sensitivity",
    "stress_related_comp",
    "fear_composite",
]

lagged_predictors = [
    "activity_level_num_t",
    "walk_frequency_num_t",
    "walk_without_leash_num_t",
    "offleash_dose_index_filled_t",
    "fetch_games_frequency_num_t",
]

outcome_labels = {
    "score_nonsocial_fear": "Nonsocial fear",
    "score_separation_related_problems": "Separation-related problems",
    "score_touch_sensitivity": "Touch sensitivity",
    "stress_related_comp": "Stress-related composite",
    "fear_composite": "Fear composite",
}

predictor_labels = {
    "score_nonsocial_fear_t": "Baseline nonsocial fear",
    "score_separation_related_problems_t": "Baseline separation-related prob
    "score_touch_sensitivity_t": "Baseline touch sensitivity",
    "stress_related_comp_t": "Baseline stress-related composite",
    "fear_composite_t": "Baseline fear composite",
    "age_years_t": "Age (years)",
    "activity_level_num_t": "Overall activity level",
    "walk_frequency_num_t": "Walk frequency",
    "walk_without_leash_num_t": "Walking without leash",
    "offleash_dose_index_filled_t": "Off-leash activity index",
    "fetch_games_frequency_num_t": "Fetch-game frequency",
}

lagged_results = []
lagged_models = []
lagged_failures = []

# Build lagged train/test splits using the existing dog-level split
lagged_train = lagged_analysis_df[
    lagged_analysis_df["subject_id"].isin(train_dogs)
].copy()

lagged_test = lagged_analysis_df[
    lagged_analysis_df["subject_id"].isin(test_dogs)
].copy()

for outcome in lagged_outcomes:
    outcome_t = f"{outcome}_t"
    outcome_next = f"{outcome}_next"

    model_vars = [
        "subject_id",
```

```
        "year_in_study_next",
        "age_years_t",
        outcome_t,
        outcome_next,
    ] + lagged_predictors

    df_model = lagged_train[model_vars].dropna().copy()

    formula = (
        f"{outcome_next} ~ {outcome_t} + age_years_t + C(year_in_study_next)
        + " + ".join(lagged_predictors)
    )

    fit = None
    optimizer_used = None

    for method in ["powell", "cg", "lbfgs"]:
        try:
            with warnings.catch_warnings():
                warnings.simplefilter("ignore")
                model = smf.mixedlm(
                    formula=formula,
                    data=df_model,
                    groups=df_model["subject_id"]
                )
                fit = model.fit(reml=False, method=method, maxiter=300)
            optimizer_used = method
            break
        except Exception as e:
            last_error = str(e)
            continue

    if fit is not None:
        lagged_models.append({
            "outcome": outcome,
            "fit": fit,
            "optimizer": optimizer_used,
            "n_rows": df_model.shape[0],
        })

        params = fit.params
        pvalues = fit.pvalues
        conf_int = fit.conf_int()

        for pred in [outcome_t, "age_years_t"] + lagged_predictors:
            if pred in params.index:
                lagged_results.append({
                    "Outcome": outcome_labels[outcome],
                    "Predictor": predictor_labels[pred],
                    "Coefficient": params[pred],
                    "p-value": pvalues[pred],
                    "CI lower": conf_int.loc[pred, 0],
                    "CI upper": conf_int.loc[pred, 1],
                    "Rows used": df_model.shape[0],
                    "Optimizer": optimizer_used,
                })
```

```
    else:
        lagged_failures.append({
            "Outcome": outcome_labels[outcome],
            "Rows used": df_model.shape[0],
            "Reason model failed": last_error,
        })

lagged_results_df = pd.DataFrame(lagged_results).round(4)

display(Markdown(
    "**Table 11.** Lagged mixed-effects model coefficients predicting next-y
))
display(lagged_results_df)

if lagged_failures:
    lagged_failures_df = pd.DataFrame(lagged_failures)
    display(Markdown(
        "The lagged mixed-effects model shown below did not converge with th
    ))
    display(lagged_failures_df)

# Save quietly
lagged_results_df.to_csv("step12_lagged_mixed_models.csv", index=False)
if lagged_failures:
    lagged_failures_df.to_csv("step12_lagged_model_failures.csv", index=Fals
```

Table 11. Lagged mixed-effects model coefficients predicting next-year behavioral outcomes from prior outcome level, walking/activity predictors, age, and study year.

	Outcome	Predictor	Coefficient	p-value	CI lower	CI upper	Rows used	Optimizer
0	Nonsocial fear	Baseline nonsocial fear	0.7168	0.0000	0.6791	0.7544	3600	powell
1	Nonsocial fear	Age (years)	-0.0038	0.7642	-0.0290	0.0213	3600	powell
2	Nonsocial fear	Overall activity level	-0.0412	0.0031	-0.0685	-0.0138	3600	powell
3	Nonsocial fear	Walk frequency	-0.0007	0.8385	-0.0078	0.0063	3600	powell
4	Nonsocial fear	Walking without leash	-0.0590	0.0746	-0.1238	0.0059	3600	powell
5	Nonsocial fear	Off-leash activity index	0.0270	0.3788	-0.0332	0.0872	3600	powell
6	Nonsocial fear	Fetch-game frequency	0.0039	0.7733	-0.0227	0.0305	3600	powell
7	Separation-related problems	Baseline separation-related problems	0.5205	0.0000	0.4756	0.5653	3523	powell
8	Separation-related problems	Age (years)	0.0095	0.2791	-0.0077	0.0266	3523	powell
9	Separation-related problems	Overall activity level	-0.0127	0.1820	-0.0313	0.0059	3523	powell
10	Separation-related problems	Walk frequency	-0.0013	0.5975	-0.0061	0.0035	3523	powell
11	Separation-related problems	Walking without leash	-0.0198	0.3782	-0.0640	0.0243	3523	powell
12	Separation-related problems	Off-leash activity index	0.0089	0.6718	-0.0321	0.0498	3523	powell
13	Separation-related problems	Fetch-game frequency	-0.0049	0.5926	-0.0231	0.0132	3523	powell
14	Touch sensitivity	Baseline touch sensitivity	0.6740	0.0000	0.6294	0.7186	3508	powell

	Outcome	Predictor	Coefficient	p-value	CI lower	CI upper	Rows used	Optimizer
15	Touch sensitivity	Age (years)	-0.0116	0.3156	-0.0343	0.0111	3508	powell
16	Touch sensitivity	Overall activity level	-0.0341	0.0069	-0.0588	-0.0094	3508	powell
17	Touch sensitivity	Walk frequency	0.0026	0.4252	-0.0038	0.0090	3508	powell
18	Touch sensitivity	Walking without leash	0.0227	0.4484	-0.0360	0.0814	3508	powell
19	Touch sensitivity	Off-leash activity index	-0.0221	0.4278	-0.0766	0.0325	3508	powell
20	Touch sensitivity	Fetch-game frequency	-0.0109	0.3785	-0.0352	0.0134	3508	powell
21	Stress-related composite	Baseline stress-related composite	0.7155	0.0000	0.6806	0.7504	3660	powell
22	Stress-related composite	Age (years)	-0.0026	0.7361	-0.0175	0.0123	3660	powell
23	Stress-related composite	Overall activity level	-0.0243	0.0033	-0.0405	-0.0081	3660	powell
24	Stress-related composite	Walk frequency	-0.0001	0.9785	-0.0042	0.0041	3660	powell
25	Stress-related composite	Walking without leash	-0.0189	0.3361	-0.0575	0.0196	3660	powell
26	Stress-related composite	Off-leash activity index	0.0039	0.8322	-0.0320	0.0398	3660	powell
27	Stress-related composite	Fetch-game frequency	-0.0040	0.6200	-0.0198	0.0118	3660	powell
28	Fear composite	Baseline fear composite	0.7121	0.0000	0.6717	0.7526	3642	powell
29	Fear composite	Age (years)	-0.0090	0.3423	-0.0275	0.0095	3642	powell

	Outcome	Predictor	Coefficient	p-value	CI lower	CI upper	Rows used	Optimizer
30	Fear composite	Overall activity level	-0.0188	0.0670	-0.0389	0.0013	3642	powell
31	Fear composite	Walk frequency	-0.0024	0.3514	-0.0076	0.0027	3642	powell
32	Fear composite	Walking without leash	-0.0232	0.3422	-0.0710	0.0247	3642	powell
33	Fear composite	Off-leash activity index	0.0045	0.8421	-0.0399	0.0490	3642	powell
34	Fear composite	Fetch-game frequency	0.0051	0.6056	-0.0144	0.0246	3642	powell

The lagged mixed models showed strong longitudinal continuity across all outcomes: prior outcome level at time t was the strongest predictor of the same outcome at time t+1.

After controlling for baseline outcome, age, and study wave, higher activity level at time t was associated with lower subsequent nonsocial fear, lower touch sensitivity, and lower stress-related composite scores, with a similar but weaker pattern for fear composite. In contrast, walk frequency, walking without leash, and the off-leash activity index did not show strong independent lagged associations in these adjusted models. Separation-related problems did not show clear lagged associations with the routine predictors.

In [36]:
```
# Compact visualization of lagged model coefficients

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

lagged_plot_df = lagged_results_df.copy()

predictor_order = [
    "Overall activity level",
    "Walk frequency",
    "Walking without leash",
    "Off-leash activity index",
    "Fetch-game frequency",
]

outcome_order = [
    "Nonsocial fear",
    "Separation-related problems",
    "Touch sensitivity",
```

```
    "Stress-related composite",
    "Fear composite",
]

coef_heatmap = (
    lagged_plot_df[lagged_plot_df["Predictor"].isin(predictor_order)]
    .pivot(index="Outcome", columns="Predictor", values="Coefficient")
    .reindex(index=outcome_order, columns=predictor_order)
)

pval_table = (
    lagged_plot_df[lagged_plot_df["Predictor"].isin(predictor_order)]
    .pivot(index="Outcome", columns="Predictor", values="p-value")
    .reindex(index=outcome_order, columns=predictor_order)
)

def p_to_stars(p):
    if pd.isna(p):
        return ""
    elif p < 0.001:
        return "***"
    elif p < 0.01:
        return "**"
    elif p < 0.05:
        return "*"
    else:
        return ""

fig, ax = plt.subplots(figsize=(10, 5.5))
im = ax.imshow(coef_heatmap.values, aspect="auto")

ax.set_xticks(np.arange(len(coef_heatmap.columns)))
ax.set_xticklabels(coef_heatmap.columns, rotation=45, ha="right")
ax.set_yticks(np.arange(len(coef_heatmap.index)))
ax.set_yticklabels(coef_heatmap.index)

for i in range(coef_heatmap.shape[0]):
    for j in range(coef_heatmap.shape[1]):
        coef = coef_heatmap.iloc[i, j]
        pval = pval_table.iloc[i, j]
        if pd.notna(coef):
            stars = p_to_stars(pval)
            ax.text(j, i, f"{coef:.3f}{stars}", ha="center", va="center")

ax.set_title("Lagged model coefficients for routine predictors")
cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Coefficient")

plt.tight_layout()
plt.show()

display(Markdown(
    "**Figure 9.** Heatmap of lagged mixed-effects model coefficients for th
))
```

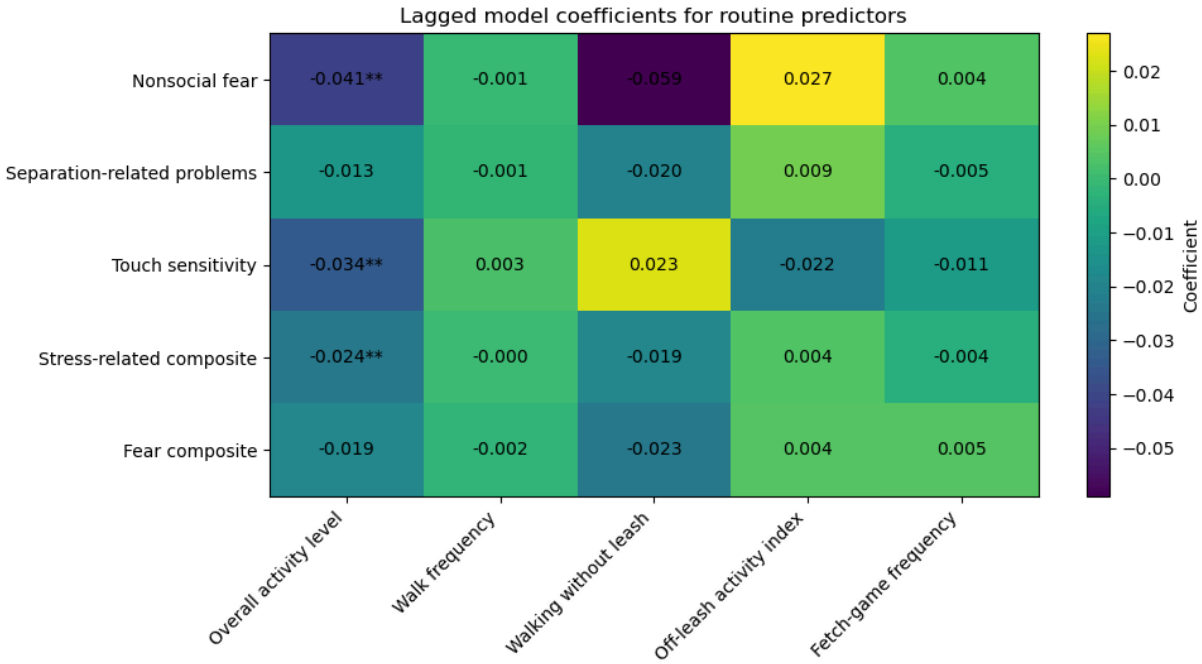


Figure 9. Heatmap of lagged mixed-effects model coefficients for the main behavioral outcomes and walking/activity predictors.

Step 13. Validation of the lagged mixed models

This section evaluates how well the lagged models generalize to an independent validation subset of dogs.

The goal is to:

- generate predictions on held-out dogs
- quantify model performance outside the calibration subset
- compare observed and predicted values using simple validation metrics
- determine whether the lagged models capture meaningful predictive signal beyond baseline stability alone

In [33]:
```
# Step 13. Validation of the lagged mixed models

import numpy as np
import pandas as pd
from sklearn.metrics import mean_squared_error, r2_score
from IPython.display import display, Markdown

validation_results = []
validation_failures = []

outcome_labels = {
    "score_nonsocial_fear": "Nonsocial fear",
    "score_separation_related_problems": "Separation-related problems",
    "score_touch_sensitivity": "Touch sensitivity",
    "stress_related_comp": "Stress-related composite",
```

```
    "fear_composite": "Fear composite",
}

for model_info in lagged_models:
    outcome = model_info["outcome"]
    fit = model_info["fit"]

    outcome_t = f"{outcome}_t"
    outcome_next = f"{outcome}_next"

    model_vars = [
        "subject_id",
        "year_in_study_next",
        "age_years_t",
        outcome_t,
        outcome_next,
    ] + lagged_predictors

    df_test = lagged_test[model_vars].dropna().copy()

    try:
        y_true = df_test[outcome_next]
        y_pred = fit.predict(df_test)

        r2 = r2_score(y_true, y_pred)
        mse = mean_squared_error(y_true, y_pred)
        rmse = np.sqrt(mse)

        denom = np.sum((y_true - np.mean(y_true)) ** 2)
        nse = 1 - np.sum((y_true - y_pred) ** 2) / denom if denom != 0 else

        validation_results.append({
            "Outcome": outcome_labels[outcome],
            "Validation rows": df_test.shape[0],
            "R²": r2,
            "RMSE": rmse,
            "NSE": nse,
        })

    except Exception as e:
        validation_failures.append({
            "Outcome": outcome_labels[outcome],
            "Reason validation failed": str(e),
        })

validation_results_df = pd.DataFrame(validation_results).round(4)

display(Markdown(
    "**Table 12.** Out-of-sample validation metrics for the lagged mixed-eff
))
display(validation_results_df)

if validation_failures:
    validation_failures_df = pd.DataFrame(validation_failures)
    display(Markdown(
        "The model shown below could not be evaluated on the held-out test s
```

```
    ))
    display(validation_failures_df)

validation_results_df.to_csv("step13_validation_results.csv", index=False)
```

Table 12. Out-of-sample validation metrics for the lagged mixed-effects models on the held-out dog-level test set.

	Outcome	Validation rows	R^2	RMSE	NSE
0	Nonsocial fear	1513	0.4934	0.4101	0.4934
1	Separation-related problems	1495	0.3153	0.2675	0.3153
2	Touch sensitivity	1453	0.3487	0.3976	0.3487
3	Stress-related composite	1537	0.4806	0.2454	0.4806
4	Fear composite	1532	0.4832	0.3152	0.4832

The lagged mixed models showed moderate held-out predictive performance on independent dogs, with validation `R²` values ranging from about 0.32 to 0.49 across outcomes.

This indicates that the lagged models captured meaningful longitudinal signal that generalized beyond the calibration subset. Much of this predictive performance reflects the strong stability of the outcomes over time, since prior outcome level at time `t` was the dominant predictor of the same outcome at time `t+1`. The routine predictors therefore should be interpreted as modest additional contributors rather than primary drivers of prediction.

Step 14. Prediction-based sensitivity analysis for walk and activity predictors

At this step, I assess how sensitive the main longitudinal models are to changes in the walk- and activity-related predictors included in the analysis.

The logic is simple: one predictor is varied at a time, while all remaining predictors are held constant at typical values. This makes it possible to see which aspects of the dog's activity and walking routine produce the largest directional shifts in the model-predicted next-year behavioral outcomes.

For ordinal predictors, the contrast is defined from the observed minimum to maximum. For the continuous composite index of off-leash activity, the contrast is defined from the 25th to the 75th percentile.

The analysis is applied to the following next-year outcomes: [score_nonsocial_fear_{t+1}] [score_separation_related_problems_{t+1}] [

score_touch_sensitivity_{t+1}] [stress_related_comp_{t+1}] and [fear_composite_{t+1}]

The goal of this step is not to make causal claims, but to identify which activity- and walk-related predictors produce the largest changes in the model-predicted future outcomes.

In [34]:
```
# Step 14. Full single-parameter sensitivity analysis for all walk/activity

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

# ---------------------------------------------------
# 1. Define outcomes and substantive predictors
# ---------------------------------------------------
sensitivity_outcomes = [m["outcome"] for m in lagged_models]

predictor_specs = {
    "activity_level_num_t": {
        "type": "ordinal",
        "contrast_label": "min to max",
    },
    "walk_frequency_num_t": {
        "type": "ordinal",
        "contrast_label": "min to max",
    },
    "walk_without_leash_num_t": {
        "type": "ordinal",
        "contrast_label": "min to max",
    },
    "offleash_dose_index_filled_t": {
        "type": "continuous",
        "contrast_label": "25th to 75th percentile",
    },
    "fetch_games_frequency_num_t": {
        "type": "ordinal",
        "contrast_label": "min to max",
    },
}

outcome_labels = {
    "score_nonsocial_fear": "Nonsocial fear",
    "score_separation_related_problems": "Separation-related problems",
    "score_touch_sensitivity": "Touch sensitivity",
    "stress_related_comp": "Stress-related composite",
    "fear_composite": "Fear composite",
}

predictor_labels = {
    "activity_level_num_t": "Overall activity level",
    "walk_frequency_num_t": "Walk frequency",
    "walk_without_leash_num_t": "Walking without leash",
```

```
    "offleash_dose_index_filled_t": "Off-leash activity index",
    "fetch_games_frequency_num_t": "Fetch-game frequency",
}

# ---------------------------------------------------
# 2. Helper functions
# ---------------------------------------------------
def get_model_vars(outcome):
    outcome_t = f"{outcome}_t"
    outcome_next = f"{outcome}_next"

    return [
        "subject_id",
        "year_in_study_next",
        "age_years_t",
        outcome_t,
        outcome_next,
        "activity_level_num_t",
        "walk_frequency_num_t",
        "walk_without_leash_num_t",
        "offleash_dose_index_filled_t",
        "fetch_games_frequency_num_t",
    ]


def build_reference_row(df_model, outcome):
    outcome_t = f"{outcome}_t"

    return {
        outcome_t: df_model[outcome_t].median(),
        "age_years_t": df_model["age_years_t"].median(),
        "year_in_study_next": int(df_model["year_in_study_next"].mode().iloc
        "activity_level_num_t": df_model["activity_level_num_t"].median(),
        "walk_frequency_num_t": df_model["walk_frequency_num_t"].median(),
        "walk_without_leash_num_t": df_model["walk_without_leash_num_t"].med
        "offleash_dose_index_filled_t": df_model["offleash_dose_index_filled
        "fetch_games_frequency_num_t": df_model["fetch_games_frequency_num_t
    }


def get_contrast_values(df_model, predictor, predictor_type):
    series = df_model[predictor].dropna()

    if predictor_type == "continuous":
        low_value = float(series.quantile(0.25))
        high_value = float(series.quantile(0.75))
    else:
        low_value = float(series.min())
        high_value = float(series.max())

    return low_value, high_value


def predict_for_row(fit, row_dict):
    pred = fit.predict(pd.DataFrame([row_dict]))
    return float(pred.iloc[0] if hasattr(pred, "iloc") else pred[0])
```

```python
# ----------------------------------------------------
# 3. Run sensitivity analysis
# ----------------------------------------------------
sensitivity_results = []

for model_info in lagged_models:
    outcome = model_info["outcome"]
    fit = model_info["fit"]

    model_vars = get_model_vars(outcome)
    df_model = lagged_train[model_vars].dropna().copy()

    ref_row = build_reference_row(df_model, outcome)

    for predictor, spec in predictor_specs.items():
        low_value, high_value = get_contrast_values(
            df_model=df_model,
            predictor=predictor,
            predictor_type=spec["type"],
        )

        row_low = ref_row.copy()
        row_high = ref_row.copy()

        row_low[predictor] = low_value
        row_high[predictor] = high_value

        pred_low = predict_for_row(fit, row_low)
        pred_high = predict_for_row(fit, row_high)

        delta_abs = pred_high - pred_low
        relative_delta_pct = np.nan
        if pred_low != 0:
            relative_delta_pct = 100.0 * delta_abs / pred_low

        delta_param = high_value - low_value
        absolute_sensitivity = np.nan
        if delta_param != 0:
            absolute_sensitivity = delta_abs / delta_param

        sensitivity_results.append({
            "Outcome": outcome_labels[outcome],
            "Predictor": predictor_labels[predictor],
            "Contrast": spec["contrast_label"],
            "Low value": low_value,
            "High value": high_value,
            "Predicted low": pred_low,
            "Predicted high": pred_high,
            "Predictor range": delta_param,
            "Absolute change": delta_abs,
            "Relative change (%)": relative_delta_pct,
            "Absolute sensitivity": absolute_sensitivity,
        })
```

```
sensitivity_results_df = pd.DataFrame(sensitivity_results)

outcome_order = [outcome_labels[o] for o in sensitivity_outcomes]
predictor_order = [predictor_labels[p] for p in predictor_specs.keys()]

sensitivity_results_df["Outcome"] = pd.Categorical(
    sensitivity_results_df["Outcome"],
    categories=outcome_order,
    ordered=True,
)
sensitivity_results_df["Predictor"] = pd.Categorical(
    sensitivity_results_df["Predictor"],
    categories=predictor_order,
    ordered=True,
)

sensitivity_results_df = sensitivity_results_df.sort_values(
    ["Outcome", "Predictor"]
).reset_index(drop=True)

display(Markdown(
    "**Table 13.** Prediction-based sensitivity analysis for the main walkir
))
display(sensitivity_results_df.round(4))

# save quietly
sensitivity_results_df.to_csv("step14_full_sensitivity_summary.csv", index=F

# ---------------------------------------------------
# 4. Heatmap of absolute predicted change
# ---------------------------------------------------
heatmap_abs = sensitivity_results_df.pivot(
    index="Outcome",
    columns="Predictor",
    values="Absolute change"
).loc[outcome_order, predictor_order]

fig, ax = plt.subplots(figsize=(10, 5.5))
im = ax.imshow(heatmap_abs.values, aspect="auto")

ax.set_xticks(np.arange(len(heatmap_abs.columns)))
ax.set_xticklabels(heatmap_abs.columns, rotation=45, ha="right")
ax.set_yticks(np.arange(len(heatmap_abs.index)))
ax.set_yticklabels(heatmap_abs.index)

for i in range(heatmap_abs.shape[0]):
    for j in range(heatmap_abs.shape[1]):
        value = heatmap_abs.iloc[i, j]
        if pd.notna(value):
            ax.text(j, i, f"{value:.3f}", ha="center", va="center")

ax.set_title("Sensitivity heatmap: absolute predicted change in outcome")
cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Absolute change")

plt.tight_layout()
```

```
plt.show()

display(Markdown(
    "**Figure 10.** Heatmap of absolute predicted change in outcome across t
))

# ---------------------------------------------------
# 5. Heatmap of relative predicted change
# ---------------------------------------------------
heatmap_rel = sensitivity_results_df.pivot(
    index="Outcome",
    columns="Predictor",
    values="Relative change (%)"
).loc[outcome_order, predictor_order]

fig, ax = plt.subplots(figsize=(10, 5.5))
im = ax.imshow(heatmap_rel.values, aspect="auto")

ax.set_xticks(np.arange(len(heatmap_rel.columns)))
ax.set_xticklabels(heatmap_rel.columns, rotation=45, ha="right")
ax.set_yticks(np.arange(len(heatmap_rel.index)))
ax.set_yticklabels(heatmap_rel.index)

for i in range(heatmap_rel.shape[0]):
    for j in range(heatmap_rel.shape[1]):
        value = heatmap_rel.iloc[i, j]
        if pd.notna(value):
            ax.text(j, i, f"{value:.1f}", ha="center", va="center")

ax.set_title("Sensitivity heatmap: relative predicted change in outcome (%)"
cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Relative change (%)")

plt.tight_layout()
plt.show()

display(Markdown(
    "**Figure 11.** Heatmap of relative predicted change in outcome across t
))
```

Table 13. Prediction-based sensitivity analysis for the main walking and activity predictors across lagged mixed-effects models.

	Outcome	Predictor	Contrast	Low value	High value	Predicted low	Predicted high	Predictor range	Al
0	Nonsocial fear	Overall activity level	min to max	0.0	2.0000	0.6247	0.5424	2.0000	-
1	Nonsocial fear	Walk frequency	min to max	0.0	7.0000	0.5880	0.5829	7.0000	-
2	Nonsocial fear	Walking without leash	min to max	0.0	1.0000	0.5836	0.5246	1.0000	-
3	Nonsocial fear	Off-leash activity index	25th to 75th percentile	0.0	0.9698	0.5836	0.6098	0.9698	
4	Nonsocial fear	Fetch-game frequency	min to max	0.0	2.0000	0.5758	0.5836	2.0000	
5	Separation-related problems	Overall activity level	min to max	0.0	2.0000	0.1736	0.1483	2.0000	-
6	Separation-related problems	Walk frequency	min to max	0.0	7.0000	0.1687	0.1597	7.0000	-
7	Separation-related problems	Walking without leash	min to max	0.0	1.0000	0.1609	0.1411	1.0000	-
8	Separation-related problems	Off-leash activity index	25th to 75th percentile	0.0	0.9698	0.1609	0.1695	0.9698	
9	Separation-related problems	Fetch-game frequency	min to max	0.0	2.0000	0.1708	0.1609	2.0000	-
10	Touch sensitivity	Overall activity level	min to max	0.0	2.0000	0.3497	0.2815	2.0000	-
11	Touch sensitivity	Walk frequency	min to max	0.0	7.0000	0.3000	0.3182	7.0000	
12	Touch sensitivity	Walking without leash	min to max	0.0	1.0000	0.3156	0.3383	1.0000	
13	Touch sensitivity	Off-leash activity index	25th to 75th percentile	0.0	0.9698	0.3156	0.2942	0.9698	-
14	Touch sensitivity	Fetch-game frequency	min to max	0.0	2.0000	0.3374	0.3156	2.0000	-

	Outcome	Predictor	Contrast	Low value	High value	Predicted low	Predicted high	Predictor range	Al
15	Stress-related composite	Overall activity level	min to max	0.0	2.0000	0.3666	0.3180	2.0000	-
16	Stress-related composite	Walk frequency	min to max	0.0	7.0000	0.3426	0.3422	7.0000	-
17	Stress-related composite	Walking without leash	min to max	0.0	1.0000	0.3423	0.3233	1.0000	-
18	Stress-related composite	Off-leash activity index	25th to 75th percentile	0.0	0.9698	0.3423	0.3460	0.9698	
19	Stress-related composite	Fetch-game frequency	min to max	0.0	2.0000	0.3502	0.3423	2.0000	-
20	Fear composite	Overall activity level	min to max	0.0	2.0000	0.3746	0.3371	2.0000	-
21	Fear composite	Walk frequency	min to max	0.0	7.0000	0.3706	0.3534	7.0000	
22	Fear composite	Walking without leash	min to max	0.0	1.0000	0.3559	0.3327	1.0000	-
23	Fear composite	Off-leash activity index	25th to 75th percentile	0.0	0.9698	0.3559	0.3602	0.9698	
24	Fear composite	Fetch-game frequency	min to max	0.0	2.0000	0.3456	0.3559	2.0000	

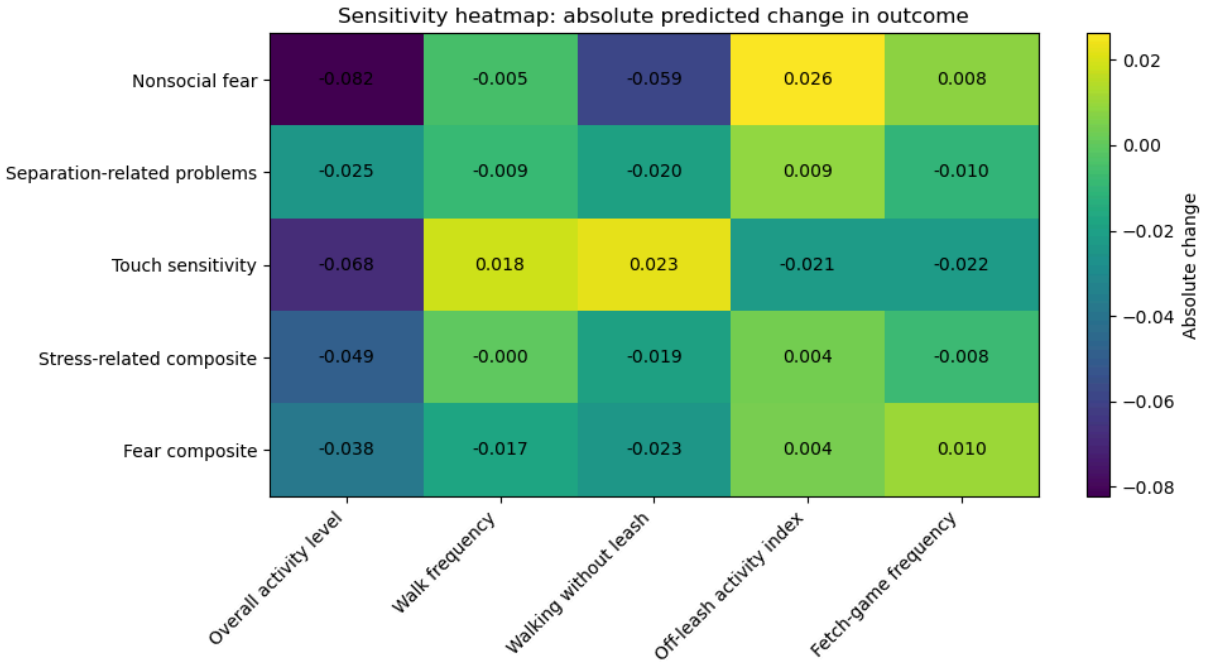


Figure 10. Heatmap of absolute predicted change in outcome across the main walking and activity predictors in the lagged mixed-effects models.

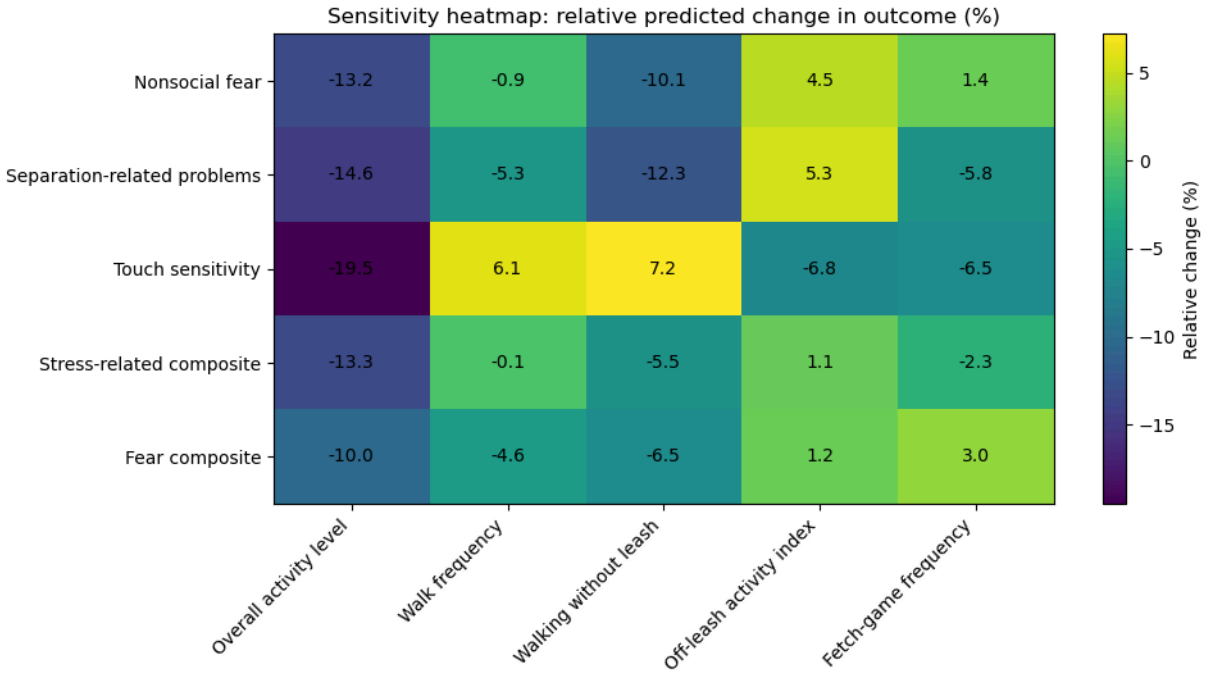


Figure 11. Heatmap of relative predicted change in outcome across the main walking and activity predictors in the lagged mixed-effects models.

Across outcomes, the clearest and most consistent sensitivity pattern was associated with overall activity level. Higher activity level was linked to lower model-predicted next-year scores for all five behavioral outcomes, suggesting that general activity level was the most influential activity-related predictor in the fitted longitudinal models.

Walking without a leash also showed a broadly similar pattern for most outcomes, but the estimated shifts were smaller and less consistent. By contrast, walking frequency

and fetch-game frequency produced only modest changes overall, with effects that were generally weak and sometimes mixed in direction.

The composite index of off-leash activity also showed relatively small and inconsistent effects across outcomes and did not indicate a clear protective pattern on its own. Taken together, these results do not support a strong universal “ideal walk” optimum defined by off-leash activity alone.

Instead, the overall pattern suggests that broader general activity level may be a more informative longitudinal correlate of later behavioral outcomes than any single walk-specific variable considered in isolation. Because these estimates are derived from observational longitudinal models, they should be interpreted as model-based associations rather than as causal effects.

Step 15. Scenario analysis for plausible activity and walking profiles

At this step, I use the fitted lagged mixed-effects models to compare a small set of plausible routine profiles that differ in overall activity level and in the composite index of off-leash activity.

Unlike the previous sensitivity analysis, which varied one predictor at a time, this step evaluates realistic predictor combinations that are easier to interpret substantively. The goal is to illustrate how the model-predicted next-year outcomes change across profiles representing lower versus higher activity and lower versus higher off-leash exposure, while the remaining predictors are held constant at typical values.

This scenario analysis is shown for two illustrative outcomes: [stress_related_comp_{t+1}] and [score_nonsocial_fear_{t+1}]

The purpose of this step is not to make causal claims, but to present the fitted model in a more interpretable profile-based form.

In [40]:
```
# Step 15. Scenario analysis for plausible routine profiles

import pandas as pd
from IPython.display import display, Markdown

# ---------------------------------------------------
# 1. Outcomes to illustrate in scenario analysis
# ---------------------------------------------------
scenario_outcomes = [
    "stress_related_comp",
    "score_nonsocial_fear",
]

# ---------------------------------------------------
```

```python
# 2. Helper functions
# ---------------------------------------------------
def get_model_vars(outcome):
    outcome_t = f"{outcome}_t"
    outcome_next = f"{outcome}_next"

    return [
        "subject_id",
        "year_in_study_next",
        "age_years_t",
        outcome_t,
        outcome_next,
        "activity_level_num_t",
        "walk_frequency_num_t",
        "walk_without_leash_num_t",
        "offleash_dose_index_filled_t",
        "fetch_games_frequency_num_t",
    ]


def build_reference_row(df_model, outcome):
    outcome_t = f"{outcome}_t"

    return {
        outcome_t: df_model[outcome_t].median(),
        "age_years_t": df_model["age_years_t"].median(),
        "year_in_study_next": int(df_model["year_in_study_next"].mode().iloc
        "activity_level_num_t": df_model["activity_level_num_t"].median(),
        "walk_frequency_num_t": df_model["walk_frequency_num_t"].median(),
        "walk_without_leash_num_t": df_model["walk_without_leash_num_t"].med
        "offleash_dose_index_filled_t": df_model["offleash_dose_index_filled
        "fetch_games_frequency_num_t": df_model["fetch_games_frequency_num_t
    }

# ---------------------------------------------------
# 3. Build plausible scenario profiles
# ---------------------------------------------------
stress_vars = get_model_vars("stress_related_comp")
stress_model_df = lagged_train[stress_vars].dropna().copy()

offleash_q25 = float(stress_model_df["offleash_dose_index_filled_t"].quantil
offleash_q75 = float(stress_model_df["offleash_dose_index_filled_t"].quantil

walk_freq_med = float(stress_model_df["walk_frequency_num_t"].median())
walk_without_leash_med = float(stress_model_df["walk_without_leash_num_t"].m
fetch_med = float(stress_model_df["fetch_games_frequency_num_t"].median())

profiles = pd.DataFrame([
    {
        "profile": "Low activity",
        "activity_level_num_t": 0,
        "offleash_dose_index_filled_t": offleash_q25,
        "walk_frequency_num_t": walk_freq_med,
        "walk_without_leash_num_t": walk_without_leash_med,
        "fetch_games_frequency_num_t": fetch_med,
    },
```

```
    {
        "profile": "Medium activity",
        "activity_level_num_t": 1,
        "offleash_dose_index_filled_t": offleash_q25,
        "walk_frequency_num_t": walk_freq_med,
        "walk_without_leash_num_t": walk_without_leash_med,
        "fetch_games_frequency_num_t": fetch_med,
    },
    {
        "profile": "High activity",
        "activity_level_num_t": 2,
        "offleash_dose_index_filled_t": offleash_q25,
        "walk_frequency_num_t": walk_freq_med,
        "walk_without_leash_num_t": walk_without_leash_med,
        "fetch_games_frequency_num_t": fetch_med,
    },
    {
        "profile": "Low activity",
        "activity_level_num_t": 0,
        "offleash_dose_index_filled_t": offleash_q75,
        "walk_frequency_num_t": walk_freq_med,
        "walk_without_leash_num_t": walk_without_leash_med,
        "fetch_games_frequency_num_t": fetch_med,
    },
    {
        "profile": "Medium activity",
        "activity_level_num_t": 1,
        "offleash_dose_index_filled_t": offleash_q75,
        "walk_frequency_num_t": walk_freq_med,
        "walk_without_leash_num_t": walk_without_leash_med,
        "fetch_games_frequency_num_t": fetch_med,
    },
    {
        "profile": "High activity",
        "activity_level_num_t": 2,
        "offleash_dose_index_filled_t": offleash_q75,
        "walk_frequency_num_t": walk_freq_med,
        "walk_without_leash_num_t": walk_without_leash_med,
        "fetch_games_frequency_num_t": fetch_med,
    },
])

# ---------------------------------------------------
# 4. Predict each outcome under each scenario
# ---------------------------------------------------
scenario_table = profiles.copy()

for model_info in lagged_models:
    outcome = model_info["outcome"]
    fit = model_info["fit"]

    if outcome not in scenario_outcomes:
        continue

    model_vars = get_model_vars(outcome)
    df_model = lagged_train[model_vars].dropna().copy()
```

```
    ref_row = build_reference_row(df_model, outcome)

    scenario_rows = []

    for _, prof in profiles.iterrows():
        row = ref_row.copy()
        row["activity_level_num_t"] = prof["activity_level_num_t"]
        row["offleash_dose_index_filled_t"] = prof["offleash_dose_index_fill
        row["walk_frequency_num_t"] = prof["walk_frequency_num_t"]
        row["walk_without_leash_num_t"] = prof["walk_without_leash_num_t"]
        row["fetch_games_frequency_num_t"] = prof["fetch_games_frequency_num
        scenario_rows.append(row)

    scenario_df = pd.DataFrame(scenario_rows)
    preds = fit.predict(scenario_df)
    scenario_table[f"predicted_{outcome}_next"] = preds.values

# ---------------------------------------------------
# 5. Build clean display table
# ---------------------------------------------------
scenario_display = scenario_table.rename(columns={
    "profile": "Scenario profile",
    "predicted_stress_related_comp_next": "Predicted stress-related composit
    "predicted_score_nonsocial_fear_next": "Predicted nonsocial fear",
}).copy()

scenario_display = scenario_display[
    [
        "Scenario profile",
        "Predicted stress-related composite",
        "Predicted nonsocial fear",
    ]
].round(4)

display(Markdown(
    "**Table 14.** Model-predicted outcomes for a set of plausible activity
))
display(scenario_display)

# save quietly
scenario_table.to_csv("step15_scenario_analysis_table.csv", index=False)
```

Table 14. Model-predicted outcomes for a set of plausible activity and off-leash routine profiles.

	Scenario profile	Predicted stress-related composite	Predicted nonsocial fear
0	Low activity	0.3666	0.6247
1	Medium activity	0.3423	0.5836
2	High activity	0.3180	0.5424
3	Low activity	0.3703	0.6510
4	Medium activity	0.3460	0.6098
5	High activity	0.3217	0.5687

In [39]:

```
# Step 15b. Visualize scenario-based predictions

import matplotlib.pyplot as plt
import numpy as np
from IPython.display import display, Markdown

plot_df = scenario_display.copy()

x = np.arange(len(plot_df))
labels = plot_df["Scenario profile"].tolist()

# ----------------------------------------
# 1. Stress-related composite
# ----------------------------------------
fig, ax = plt.subplots(figsize=(10, 5))

ax.bar(x, plot_df["Predicted stress-related composite"])

ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=30, ha="right")
ax.set_ylabel("Predicted next-year value")
ax.set_title("Scenario analysis: predicted stress-related composite")

plt.tight_layout()
plt.show()

display(Markdown(
    "**Figure 12.** Scenario-based predictions for the stress-related compos
))

# ----------------------------------------
# 2. Nonsocial fear
# ----------------------------------------
fig, ax = plt.subplots(figsize=(10, 5))

ax.bar(x, plot_df["Predicted nonsocial fear"])

ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=30, ha="right")
ax.set_ylabel("Predicted next-year value")
ax.set_title("Scenario analysis: predicted nonsocial fear")
```

```
plt.tight_layout()
plt.show()

display(Markdown(
    "**Figure 13.** Scenario-based predictions for nonsocial fear across pla
))
```

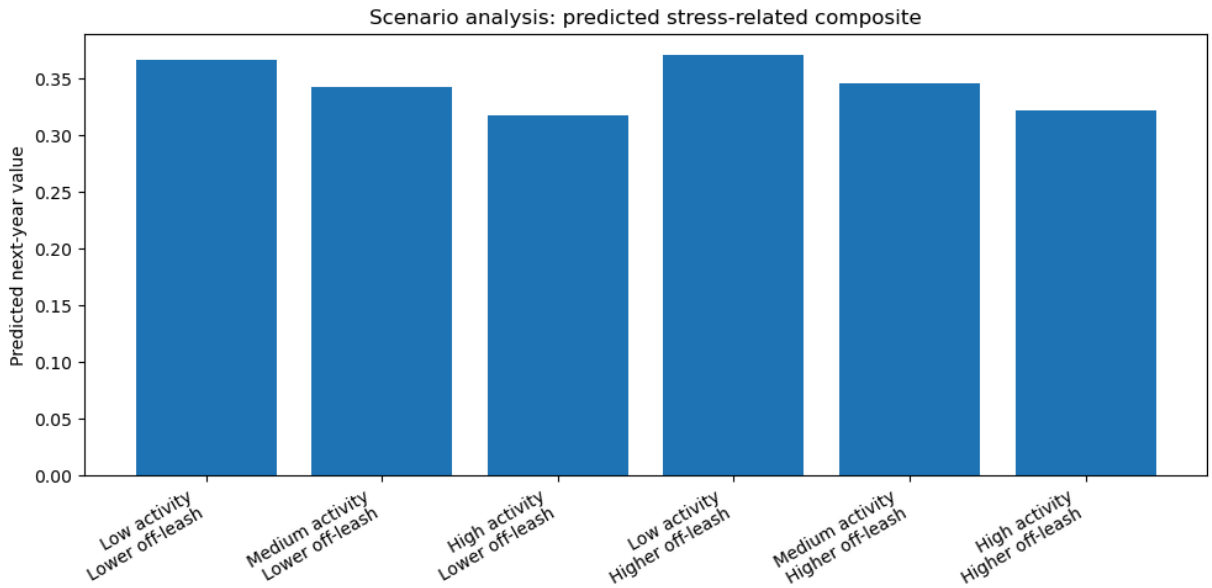


Figure 12. Scenario-based predictions for the stress-related composite across plausible activity and off-leash routine profiles.

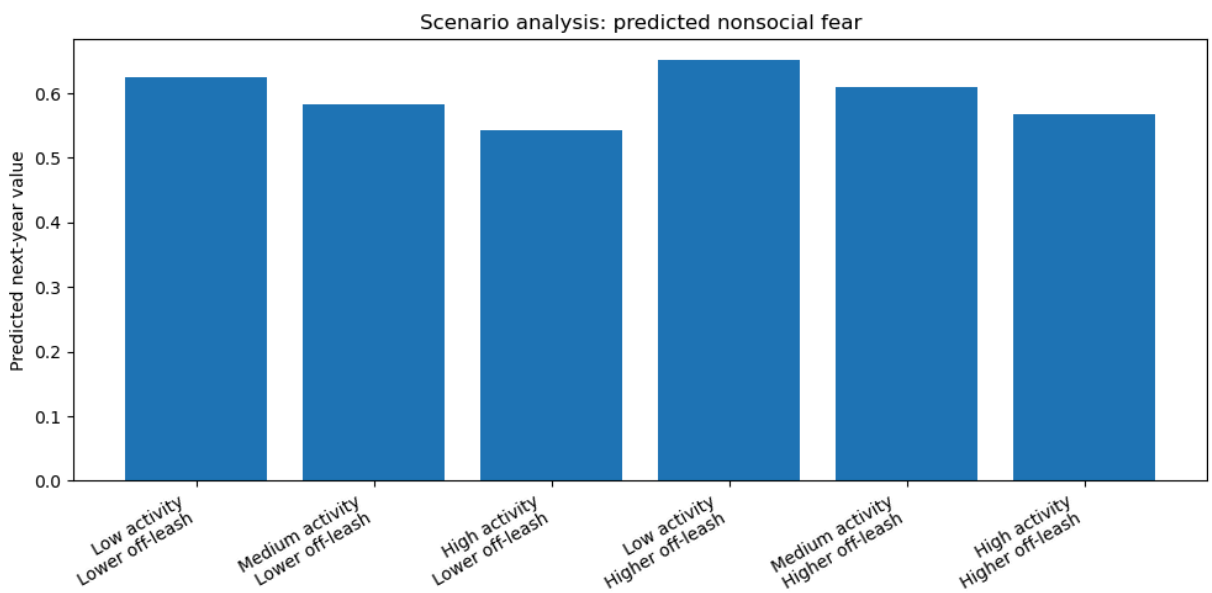


Figure 13. Scenario-based predictions for nonsocial fear across plausible activity and off-leash routine profiles.

The scenario analysis showed a clear and consistent pattern across both illustrative outcomes. As overall activity level increased from low to medium to high, the model-predicted next-year values decreased in an orderly way for both stress-related behavior and nonsocial fear. This pattern was present under both lower and higher levels of off-leash activity.

By contrast, increasing the composite index of off-leash activity did not improve the predicted outcomes within a given activity level. For the same low, medium, or high activity profile, the higher off-leash scenario produced slightly higher predicted values for both outcomes.

Taken together, these profile-based results support the same general conclusion as the sensitivity analysis: broader overall activity level appears to be more informative than off-leash activity alone. At the same time, the absence of a clear beneficial pattern for higher off-leash exposure does not support a single universal “ideal walk” profile defined mainly by off-leash activity. These scenario-based predictions should be interpreted as model-based associations rather than as causal effects.

Discussion

This study addressed two main questions: how stable fear- and anxiety-related behavioral measures and walking/activity routine measures are across years in dogs, and whether annual walking and activity routines predict subsequent levels of these behavioral outcomes after accounting for prior problem level, age, and study year.

With respect to the first question, the results indicate that fear- and anxiety-related outcomes in dogs show moderate longitudinal stability, but are neither fully fixed traits nor random noise ($r \approx 0.53$-0.74, Pearson, for the 0→1 and 1→2 transitions). This was especially evident for nonsocial fear and the two composite indices. At the same time, high rank-order stability did not imply the absence of mean-level change. On the contrary, nonsocial fear, touch sensitivity, the fear composite, and the stress-related composite all showed small average increases across both yearly transitions, whereas separation-related problems showed a small average decrease; in all cases, these shifts were statistically significant (Wilcoxon signed-rank, $p \leq 0.0058$; for separation-related problems, $p \leq 0.0002$). In other words, dogs largely retained their relative positions within the sample from year to year, but the average levels of several outcomes still drifted gradually over time.

A similar pattern, though with greater plasticity, was observed for walking and activity measures. Their year-to-year stability was also moderate ($r \approx 0.46$-0.72, Pearson), but mean-level shifts were generally more pronounced. In particular, both walking frequency and overall activity level declined over time, and these changes were statistically significant in both yearly transitions (Wilcoxon signed-rank, all $p < 0.001$). This suggests that walking and activity routines may be more responsive to changing life circumstances and day-to-day organization of the dog’s routine, whereas fear- and anxiety-related outcomes form a more stable behavioral background. Against this backdrop, the lagged models made it possible to test how these more plastic routines related to more stable behavioral outcomes over time.

With respect to the second question, the lagged linear mixed-effects models, adjusted for baseline outcome, age, study year, and dog-level random intercepts, showed that the strongest predictor of future problem level was the prior level of the same outcome. Across all outcomes, baseline coefficients were large and statistically significant ($\beta \approx 0.52$-0.72, all $p < 0.001$, MixedLM). Put simply, the models relied primarily on the dog's existing level of difficulty: dogs with higher values in the current year were much more likely to remain higher in the following year.

At the same time, walking and activity routines did not disappear entirely from the models after adjustment for baseline, age, and study year. The most consistent signal was observed for overall activity level. Higher overall activity in the current year was associated with lower subsequent nonsocial fear ($\beta = -0.041$, $p = 0.003$, MixedLM), lower touch sensitivity ($\beta = -0.034$, $p = 0.007$, MixedLM), and lower stress-related composite scores ($\beta = -0.024$, $p = 0.003$, MixedLM). For the fear composite, the effect remained in the same direction but reached only trend-level significance ($\beta = -0.019$, $p = 0.067$, MixedLM). By contrast, more specific walking characteristics were generally weaker: walking frequency showed no independent lagged associations with any outcome, and off-leash activity measures were either non-significant or inconsistent in direction. Thus, to the extent that a routine-related signal was present, it appeared to reflect a broader overall level of day-to-day activity rather than any single specific feature of walking.

On the held-out validation set, the models retained moderate predictive performance ($R^2 \approx 0.32$-0.49), indicating that they captured meaningful longitudinal signal beyond the training data. Predictive performance was better for nonsocial fear and the composite outcomes than for separation-related problems, which is consistent with the greater temporal stability of those measures.

The prediction-based sensitivity analysis and the scenario analysis led to the same overall conclusion. When one predictor was varied at a time, the clearest and most consistent directional effect again involved overall activity level: moving from the minimum to the maximum activity level was associated with lower model-predicted next-year outcomes, particularly for nonsocial fear and stress-related outcomes. By contrast, predictors related to off-leash activity produced much smaller and less consistent shifts. Likewise, in the scenario analysis, moving from low to high activity level was associated with orderly reductions in predicted stress-related composite scores and nonsocial fear, whereas higher off-leash activity by itself did not produce an equally clear protective pattern. Substantively, this suggests that everyday activity and walking routines are likely related to later fear- and anxiety-related outcomes, but did not emerge as the dominant drivers of subsequent behavior.

Several limitations should be considered when interpreting these findings. First, these are observational cohort data, so the associations identified here should not be interpreted as causal effects. Second, both behavior and activity measures were based on owner report and are therefore subject to the usual limitations of questionnaire-based

assessment and potential systematic bias. Third, the analysis was restricted to a single breed, Golden Retrievers, which limits direct generalization to the broader dog population. Fourth, the interval between observations was one year, meaning that the study captures relatively coarse longitudinal change rather than short-term dynamics. Finally, the effect sizes for routine-related predictors were generally small, which calls for especially cautious practical interpretation.

An important direction for future research is to examine links between walking routines not only and owner-reported fear/anxiety outcomes, but also more directly and explicitly with markers of chronic stress. In the present study, the behavioral outcomes inevitably reflect chronic stress only indirectly. This matters because, in a related dataset, the association between C-BARQ measures and indices of chronic stress was only moderate and did not exceed $\rho = 0.45$ (Spearman). In other words, there is overlap between behavioral measures and more direct indicators of chronic stress, but it is clearly incomplete. A particularly valuable next step would therefore be to test which aspects of walking, activity, routine predictability, and day-to-day environmental experience are associated not only with owner-reported behavior, but also with more direct markers of chronic stress, using longitudinal models that more clearly separate between-dog and within-dog effects. It would also be important to evaluate whether the same general pattern is reproduced in broader multi-breed cohorts, since associations between everyday routines and emotional outcomes may differ across dogs with different behavioral predispositions, body size, lifestyle, and management context. In addition, future studies could benefit from predictors that better capture agency-related aspects of daily life, such as opportunities for choice, control over movement and pace, and exploratory access to the environment. Another important question is whether routine-related effects are heterogeneous, for example whether activity and walking variables matter differently for dogs with low versus high baseline fear or stress vulnerability.

Overall, this study suggests that fear- and anxiety-related outcomes in this cohort of dogs are moderately stable over time, whereas walking and activity measures are somewhat more plastic. Annual routines did not emerge as the leading determinants of later behavior, but neither did they reduce to pure noise: their contribution was small, yet in several models it was systematic and interpretable. The practical take-home message is that everyday activity may represent one modest but potentially manageable component of the broader system of factors that shape dogs' emotional health. For that reason, further research on walking routines, activity, and their quality matters not only for understanding fear- and anxiety-related behavior, but also for the broader goal of reducing chronic stress in urban environments and strengthening the human-animal bond.

References

ASPCA. (2024). U.S. animal shelter statistics. Retrieved September 17, 2025, from https://www.aspca.org/helping-shelters-people-pets/us-animal-shelter-statistics

Beaver, B. V. (2024). The prevalence of behavior problems in dogs in the United States. *Journal of Veterinary Behavior, 76*, 34-39. https://doi.org/10.1016/j.jveb.2024.11.001

Goold, C., & Newberry, R. C. (2017). Modelling personality, plasticity and predictability in shelter dogs. *Royal Society Open Science, 4*(9), 170618. https://doi.org/10.1098/rsos.170618

Labadie, J. M., Swafford, B., DePena, M., Tietje, K., Page, R., et al. (2022). Cohort profile: The Golden Retriever Lifetime Study (GRLS). *PLOS ONE, 17*(6). https://doi.org/10.1371/journal.pone.0269425

Li, Y., Sexton, C. L., Dog Aging Project Consortium, Fitzpatrick, A., & Ruple, A. (2025). An analysis of behavioral characteristics and enrollment year variability in 47,444 dogs entering the Dog Aging Project from 2020 to 2023. *PLOS ONE, 20*(9). https://doi.org/10.1371/journal.pone.0330257

Muth, C., Bales, K., Hinde, K., Maninger, N., Mendoza, S. P., & Ferrer, E. (2016). Alternative models for small samples in psychological research: Applying linear mixed effects models and generalized estimating equations to repeated measures data. *Educational and Psychological Measurement, 76*(1), 64-87. https://doi.org/10.1177/0013164415580432

O'Rourke, A., Haydock, R., Butterwick, R., German, A., Carson, A., Lyle, S., & O'Flynn, C. (2025). Estimated activity levels in dogs at population scale with linear and causal modeling. *Frontiers in Veterinary Science, 12*, https://doi.org/10.3389/fvets.2025.1572794

Salonen, M., Sulkama, S., Mikkola, S., Puurunen, J., Hakanen, E., Tiira, K., Araujo, C., & Lohi, H. (2020). Prevalence, comorbidity, and breed differences in canine anxiety in 13,700 Finnish pet dogs. *Scientific Reports, 10*, 2962. https://doi.org/10.1038/s41598-020-59837-z

Sergi, V., & Panizzolo, G. (2019). Does walking the dog reduce behavioral problems? *Dog Behavior, 5*, 33-36. https://doi.org/10.4454/db.v5i1.102

Tiira, K., & Lohi, H. (2014). Reliability and validity of a questionnaire survey in canine anxiety research. *Applied Animal Behaviour Science, 155*, 82-92. https://doi.org/10.1016/j.applanim.2014.03.007

Zilocchi, M., Tagliavini, Z., Cianni, E., & Gazzano, A. (2016). Effects of physical activity on dog behavior. *Dog Behavior, 2*(2), 9-14. https://doi.org/10.4454/db.v2i2.34

In [ ]:

In [ ]: